

Operating modes of 8086

How to decide operating mode of 8086 ?????

$MN/\overline{MX}$ pin of 8086

Minimum mode

- When there is only one processor it is called as minimum mode.

$$MN/\overline{MX} = 1$$

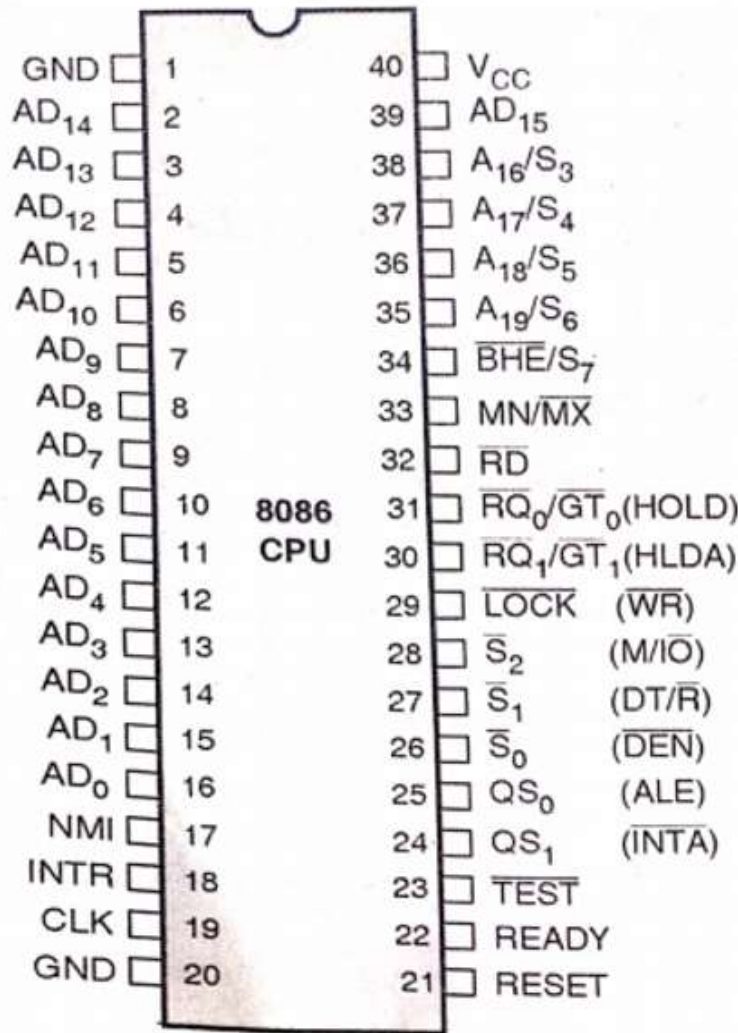
Maximum mode

- When there is more than one processor it is called as maximum mode.

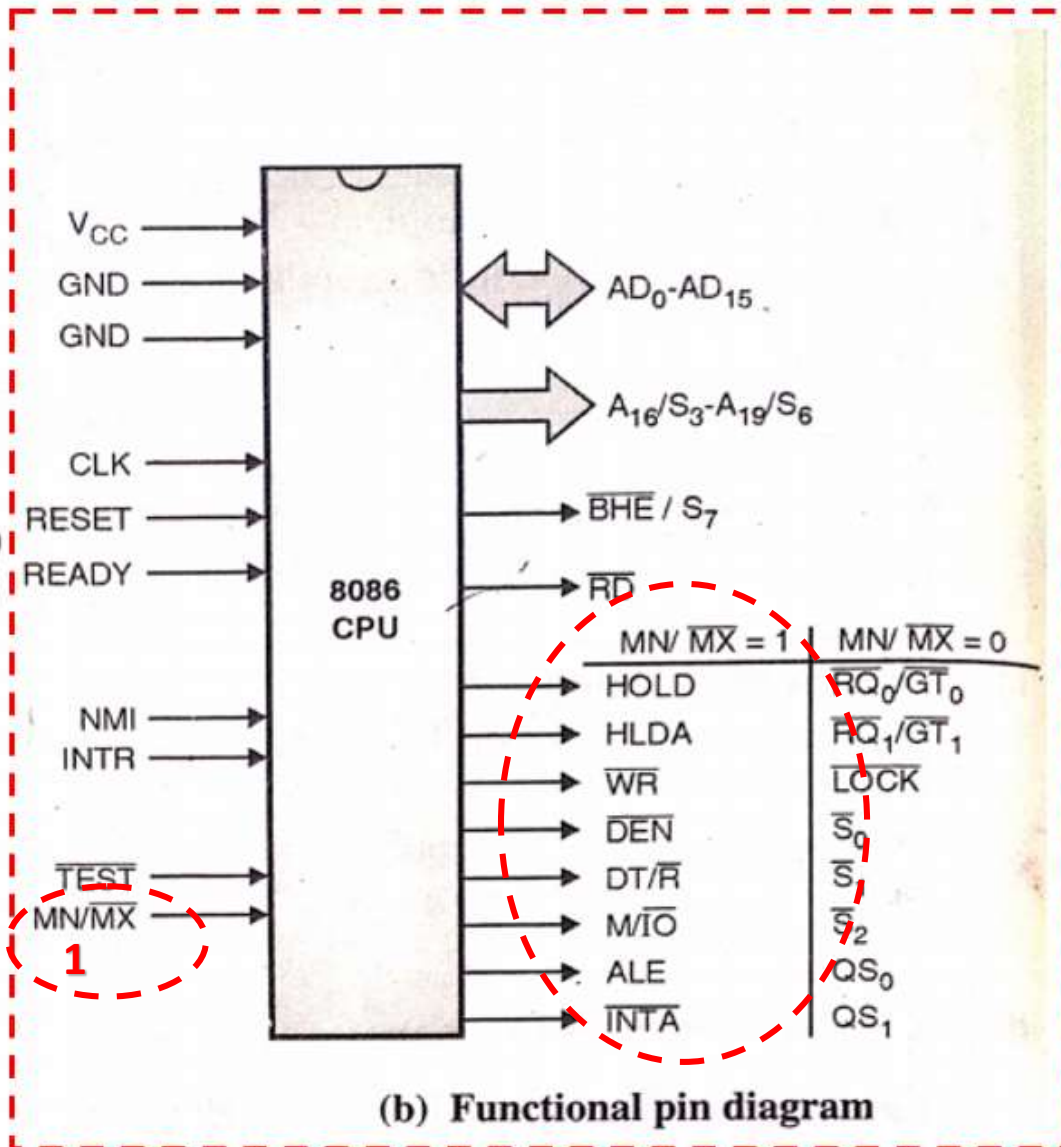
$$MN/\overline{MX} = 0$$

Minimum mode of 8086

Pin diagram of 8086



(a) Pin configuration



(b) Functional pin diagram

➡ For Exam



Minimum mode of 8086

S3-S7 are status signals given by processor

S3	S4	Seg
0	0	ES
0	1	SS
1	0	CS
1	1	DS

S3 and S4 signals use to tell right now the operation that happening, is happening on which segment

S5 : 0= IF=0
1= IF =1

S6 : 8086=Bus Master(the one who controls the bus)
Other=1

S7 : Reserved

To enable higher bank(odd)

→ BHE

S7



BHE /S7

1

A19 - A16

S6 - S3



A19/S6 - A16/S3

4

A15 - A0

D15 - D0



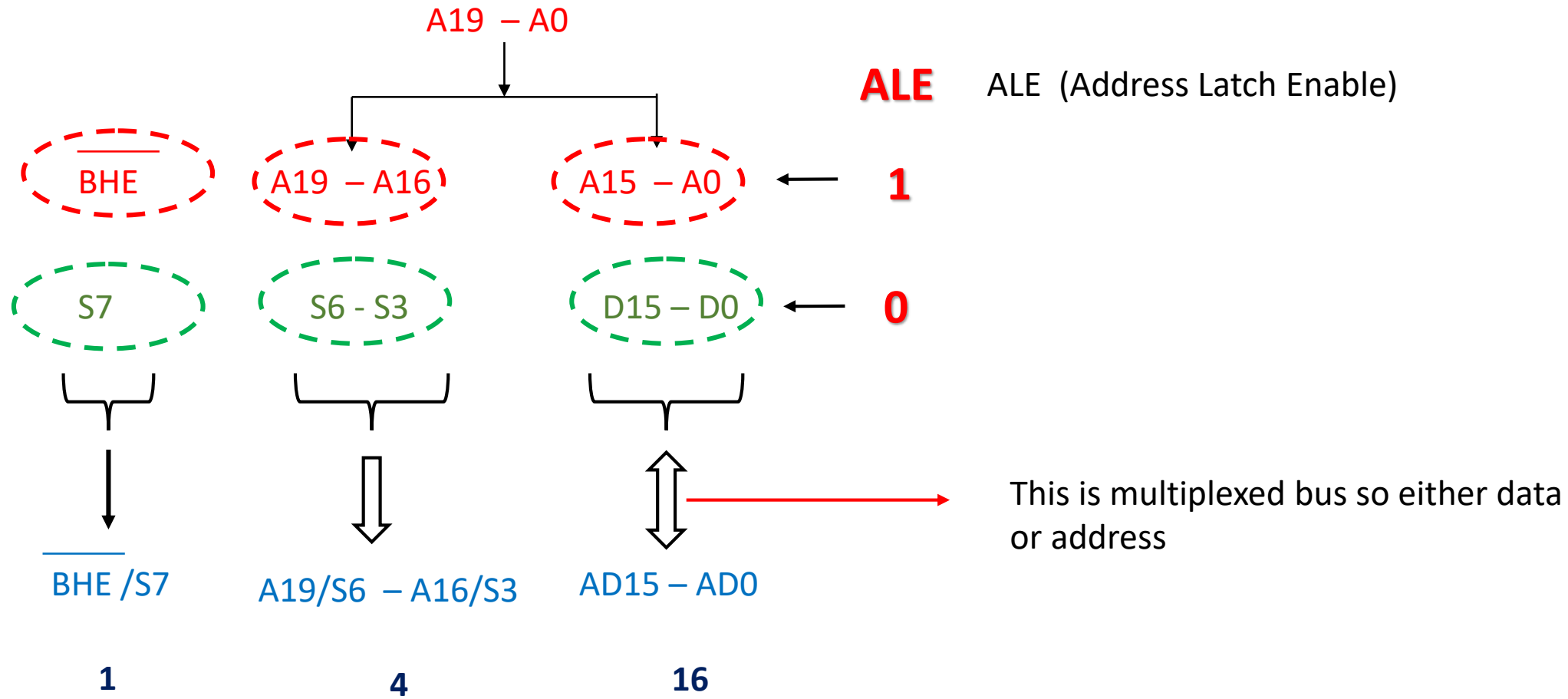
AD15 - AD0

16

← To enable lower bank(even)

21 pins are multiplexed pins

How to perform multiplexing????

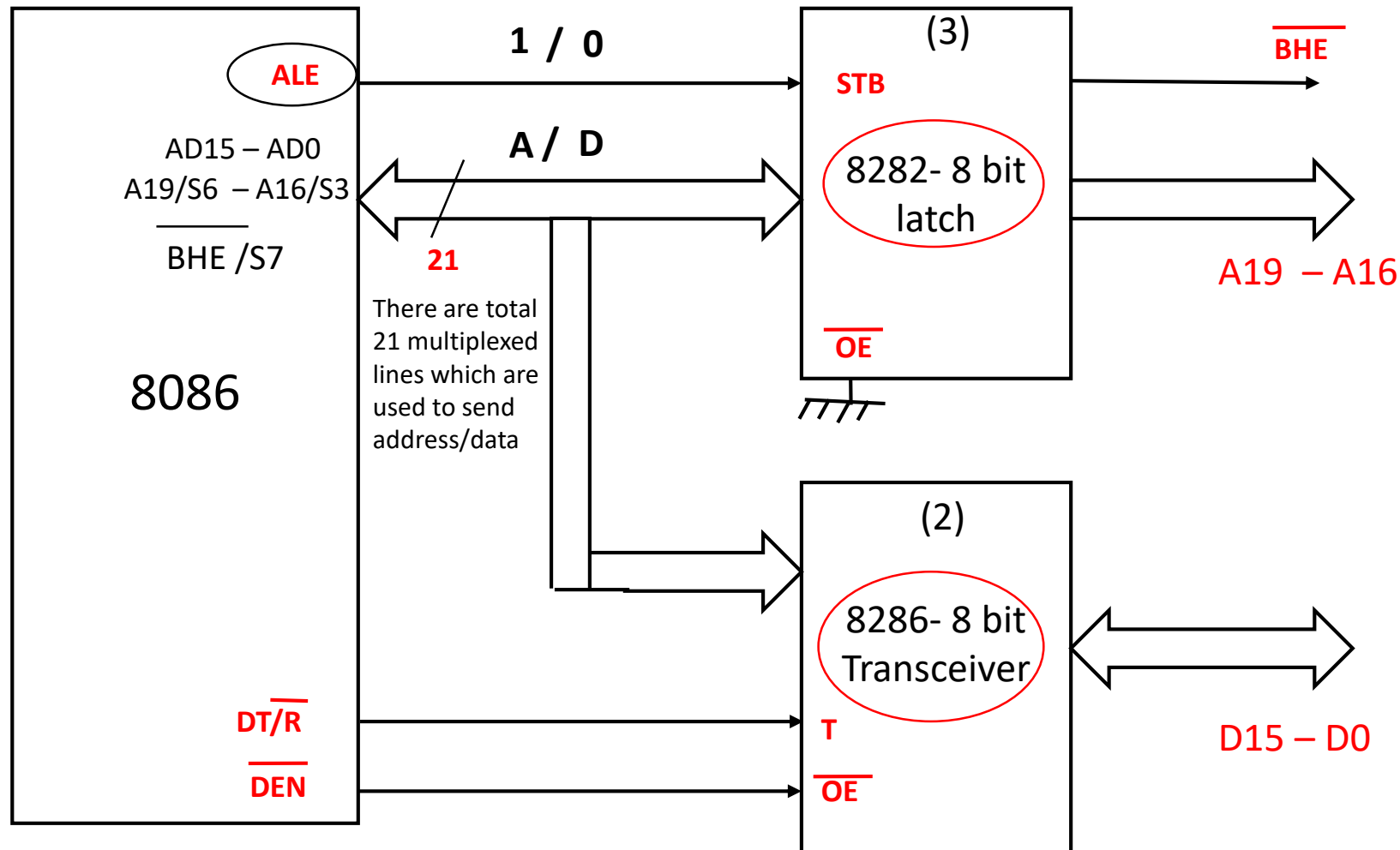


To reduce no.of pins multiplexing is used.

To differentiate between data and address Demultiplexing is used.

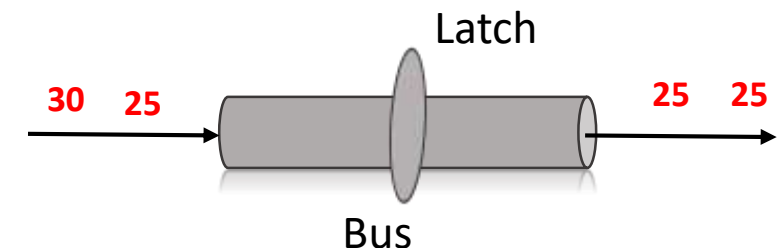
Demultiplexed Address and Data

Address bus is used to access the memory locations and data bus is used to access the data of that memory location, both are the different.
Hence demultiplexing is used.



- Bus does not store value it is used for transfer of data or address
- To store the value we required separate chips : 1. 8282 latch
2. 8286 transceiver
- Latch --- Address & Transceiver-- data

- Latch is used to store the address
- It has flip flops inside to store address
- It consist 2 signals OE i.e. output enable and STB strobe
- OE bar is grounded means it is always active low.
- Important signal is STB i.e Strobe.

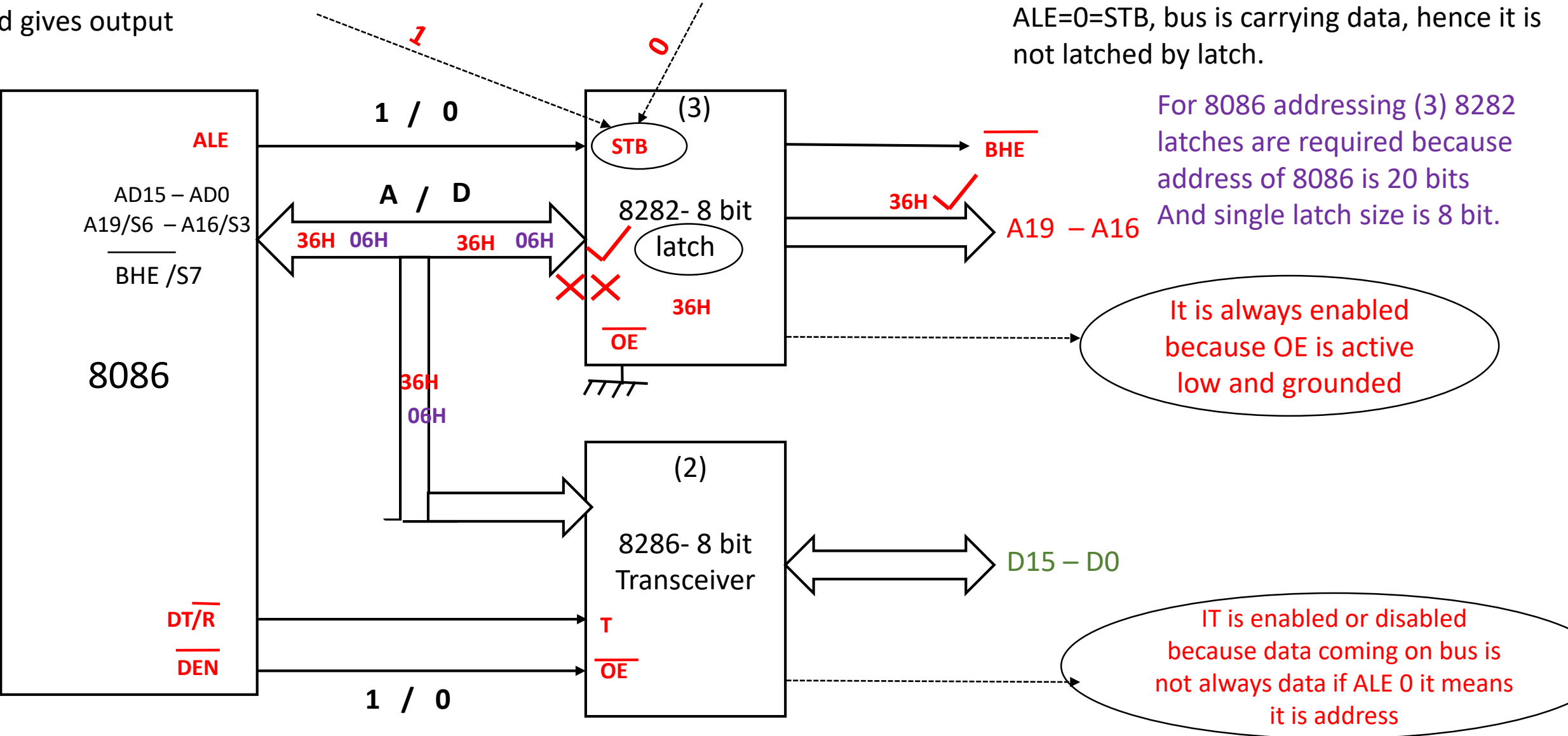


Demultiplexing of an Address Bus

When strobe is 1 whatever contents pass on bus is allowed to enter into latch and gives output

When strobe is 0 whatever contents pass on address bus is not allowed to enter into latch. It will display the previous(old) value.

ALE is connected with STB which is used to enables the latching of address.
ALE=1=STB, bus is carrying address which is latched by 8282 latch.
ALE=0=STB, bus is carrying data, hence it is not latched by latch.



Demultiplexing of Data Bus

Is in this a pure data bus???

??? NO

When ALE=1, bus carries Address, that addr will go into the latch

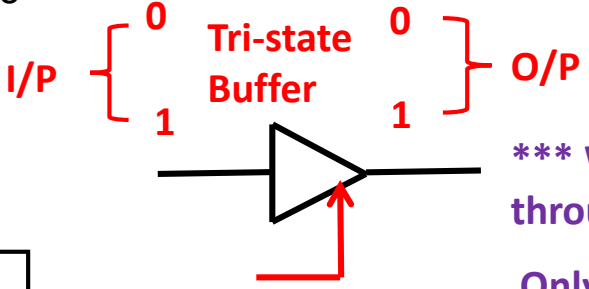
That addrs will also go down

Hence for getting pure data bus we required one more chip which is called as 8286 – transceiver (transmit and receive)

It is 8-bit tri-state buffer which is used to transmit and receive the data.

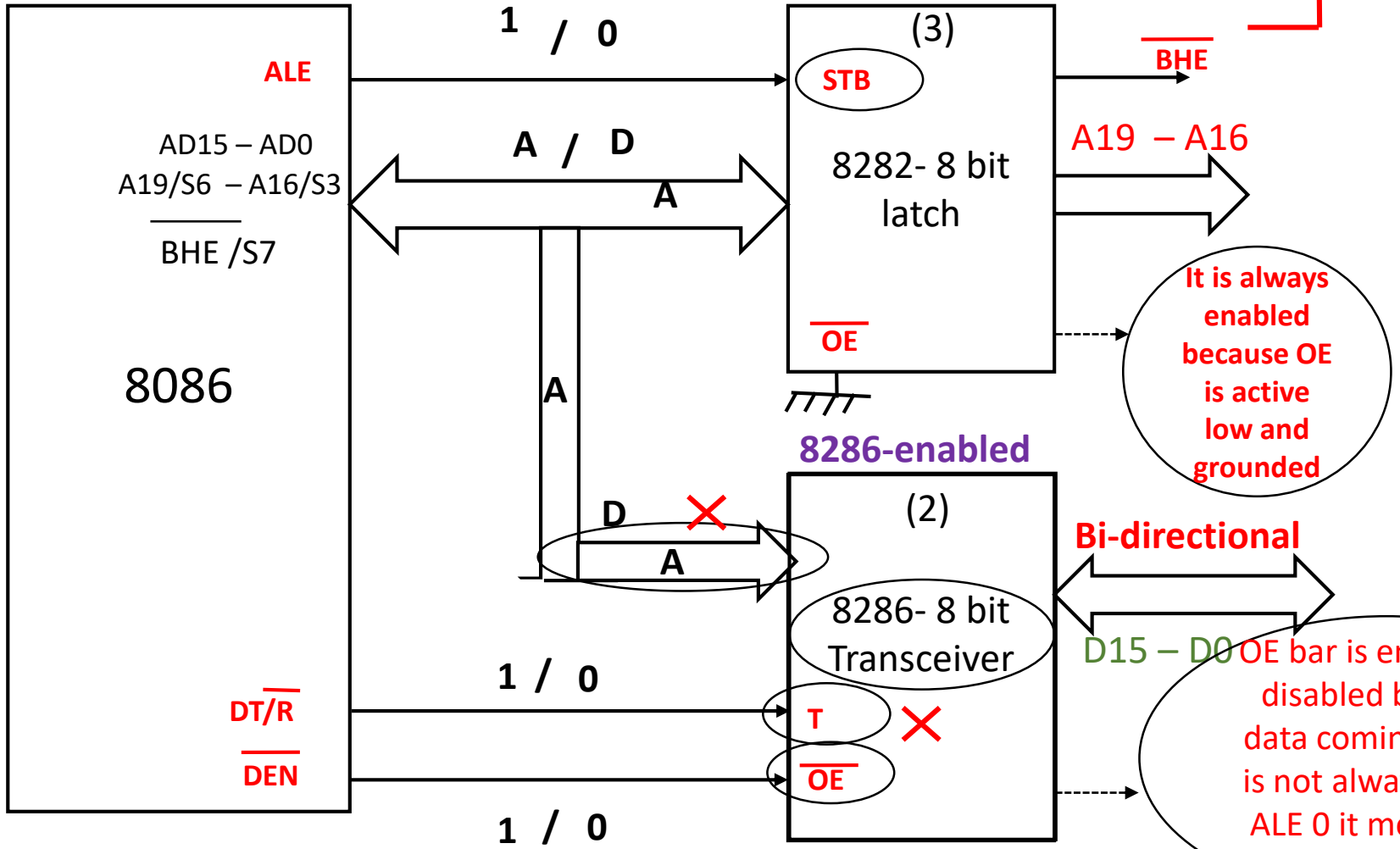
It has 2 signals: T and OE bar

When enable then only output appears on output line



*** When ALE = 0 and addr will pass through bus transceiver should disabled

Only when the bus carries data transceiver should enabled and it will pass data.



OE is connected with DEN signal of 8086.

DEN (data enable) is sends 0 when bus carries data

Transmitter or Receiver?????

T (Transmitter) is connected with DT/R signal of 8086.

DT/R = 0 , 8286 Receive data.

DT/R = 1, 8286 Transmit Data

OE bar is enabled or disabled because data coming on bus is not always data if ALE 0 it means it is address

For 8086 data (2) 8286 transceivers are required because data of 8086 is 16 Bits.

Which signal changes first????

8282 enable and latches the address

ALE- STB

1

0

8282 disable and data will not enter

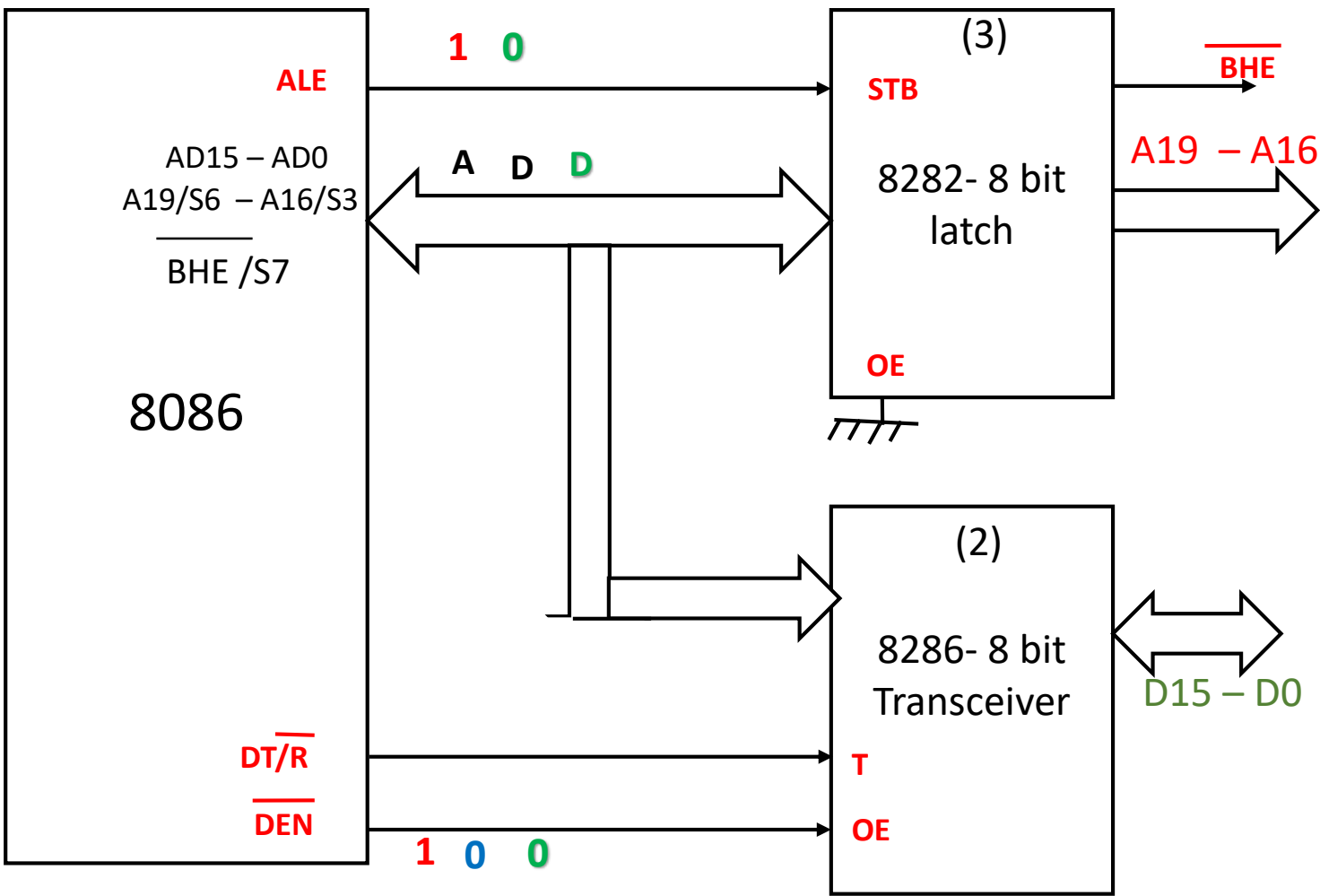
$\overline{\text{DEN}}$ - $\overline{\text{OE}}$

1

0

8286 enable and store the data

8286 disable so address will not enter



ALE = 1, bus will become address bus

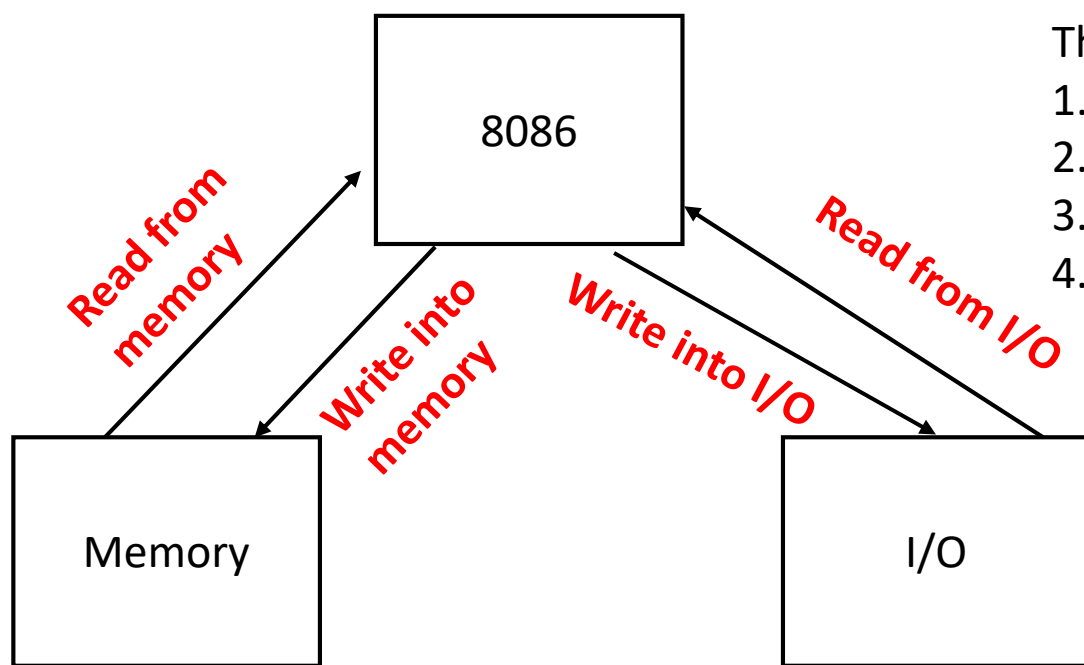
DEN = 1, bcz bus carries address

If DEN bar changes first i.e become 0 to 1 the whole logic become wrong because bus still carries address.

If bus changes then Data will also transferred into address

Sequence :

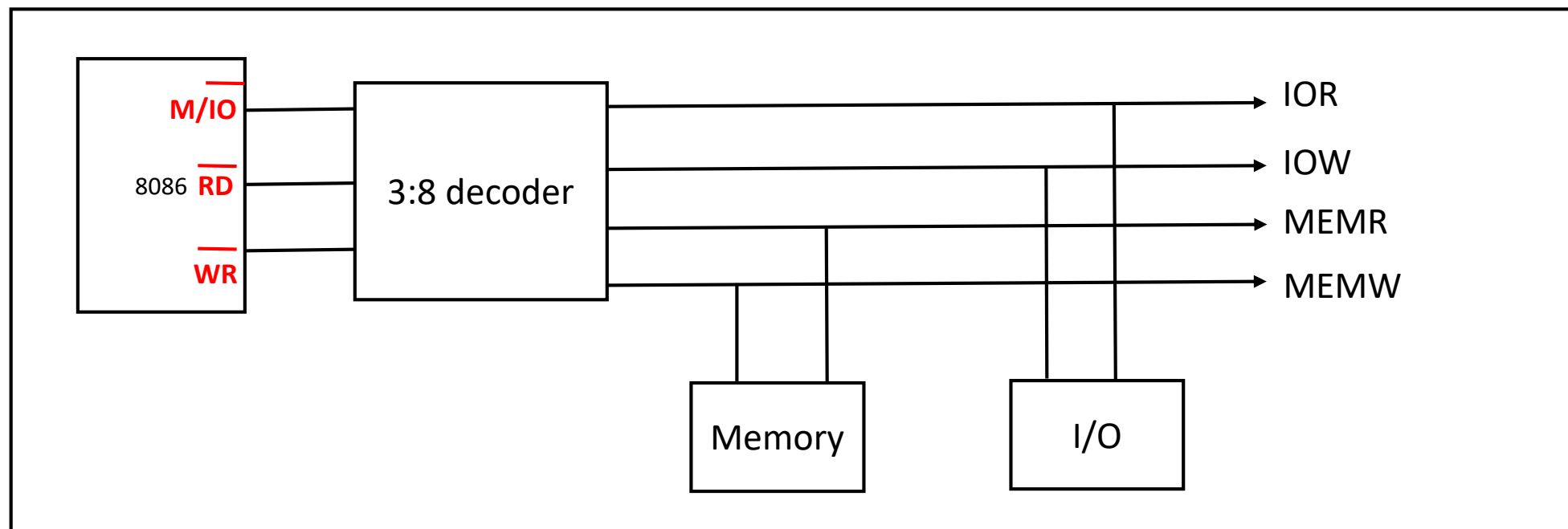
1. ALE = 1 & DEN bar=1, bus will become address bus and address latched.
2. ALE = 0 but still bus carries address, hence DEN bar=1
3. now change the mode of bus from A to D
4. make DEN bar = 0 to get the data

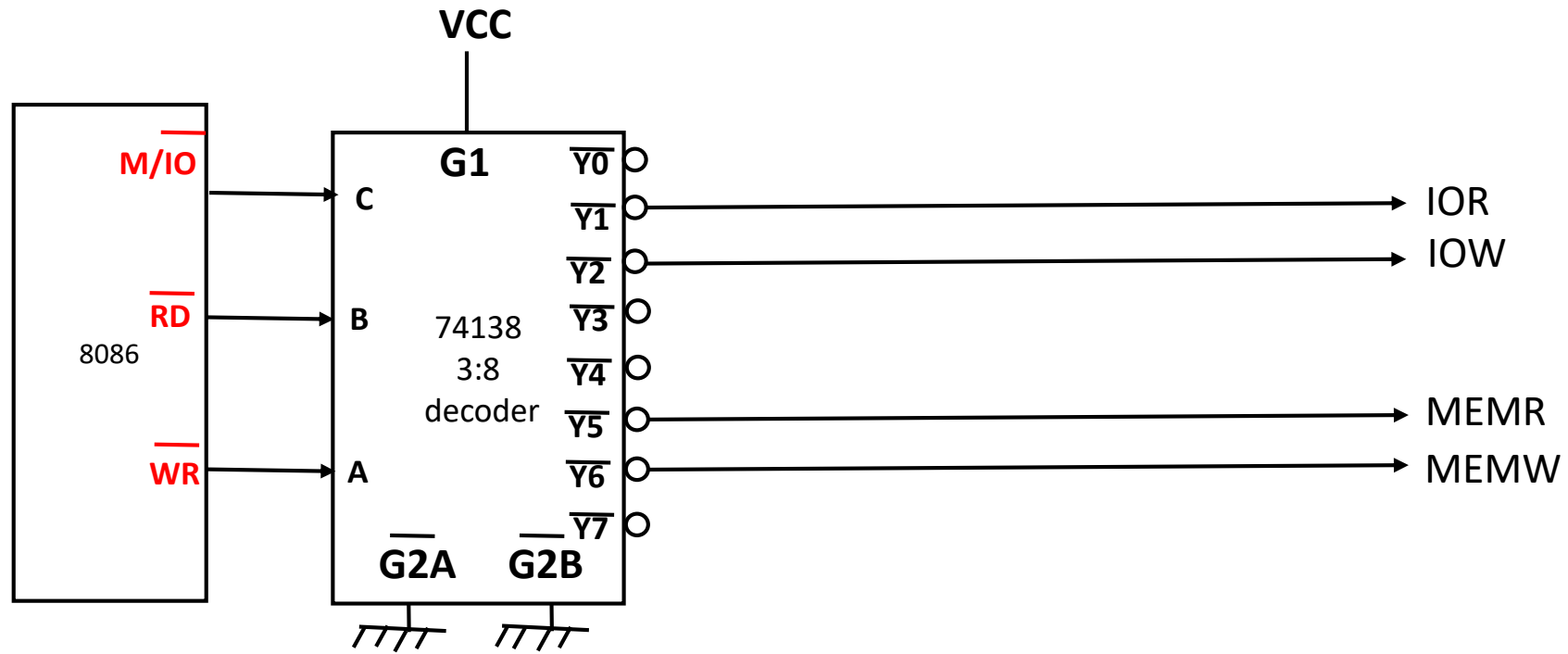


There are four operations :

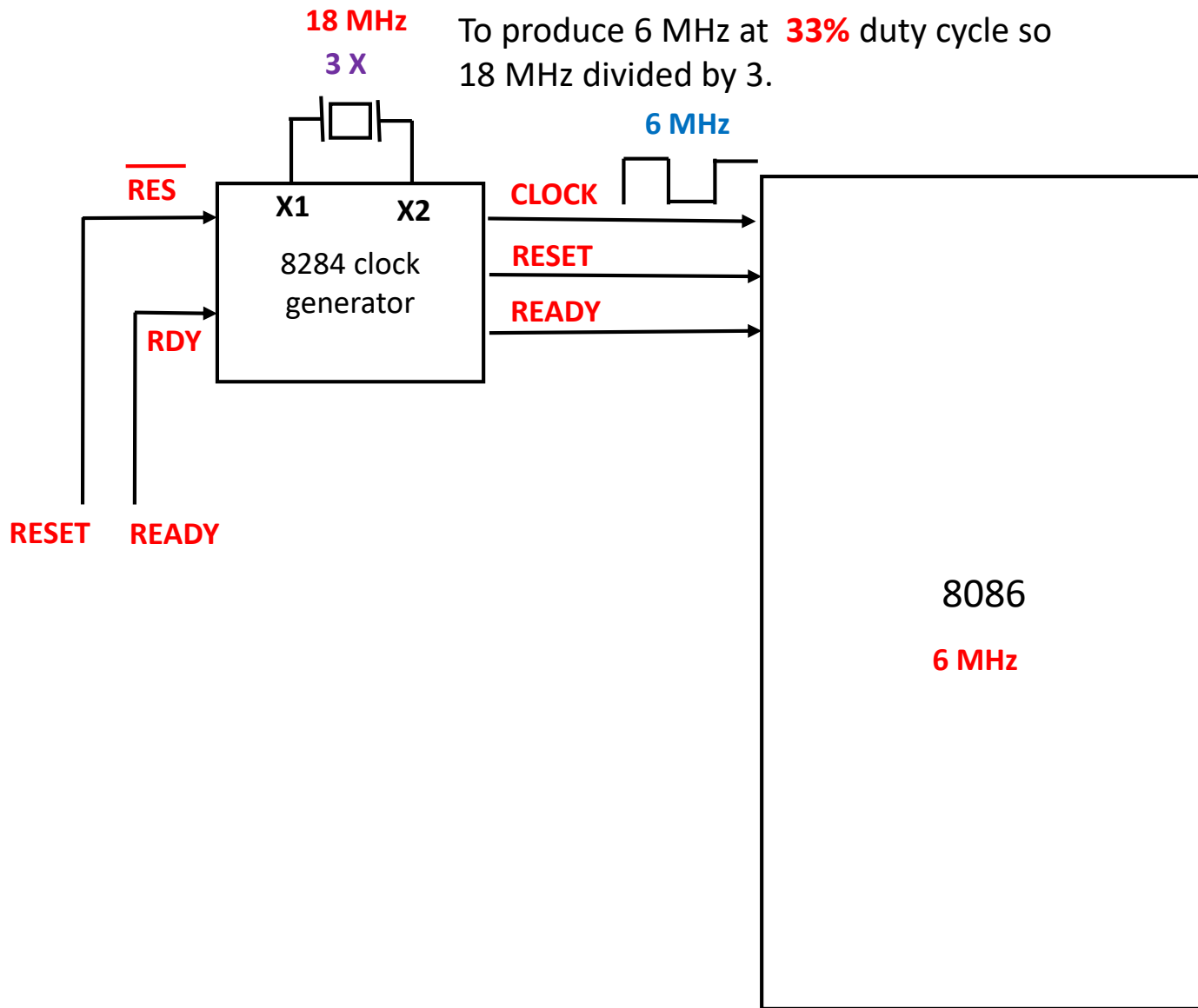
1. Memory read
2. Memory write
3. I/O read
4. I/O write

	M/ $\overline{\text{IO}}$	$\overline{\text{RD}}$	$\overline{\text{WR}}$
IOR	0	0	1
IOW	0	1	0
MEMR	1	0	1
MEMW	1	1	0



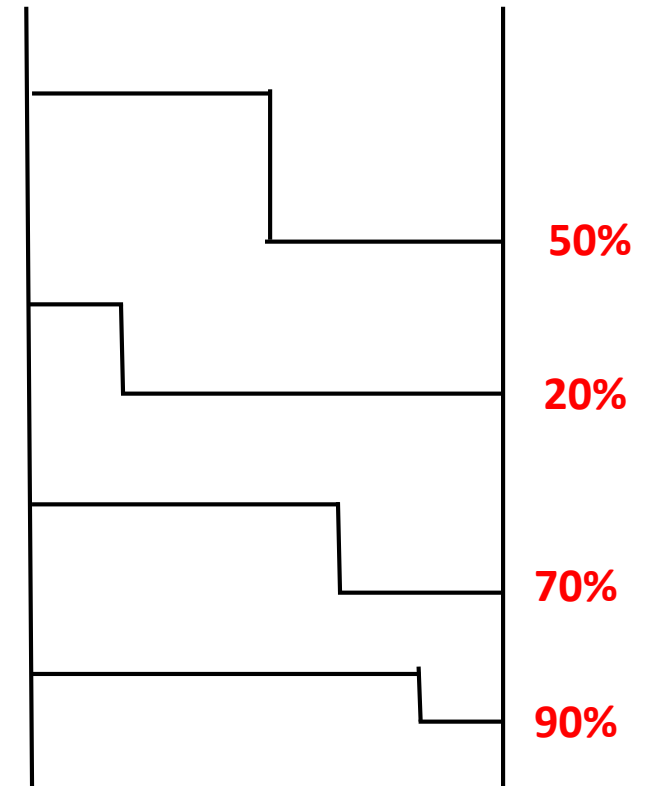


	M/IO	RD	WR
IOR	0	0	1
IOW	0	1	0
MEMR	1	0	1
MEMW	1	1	0



Duty Cycle :

1. It is the ratio of ON time and total time
2. A clock pulse is on for some time and off for some time.
3. It can be active high or active low.
4. At what time of point dose the transition take place is called as duty cycle.
5. If transition is at mid point it is called 50% duty cycle



In above example all of have different duty cycle but same clock frequency

- Each clock pulse is used to triggered processor to perform activities.

Interrupt

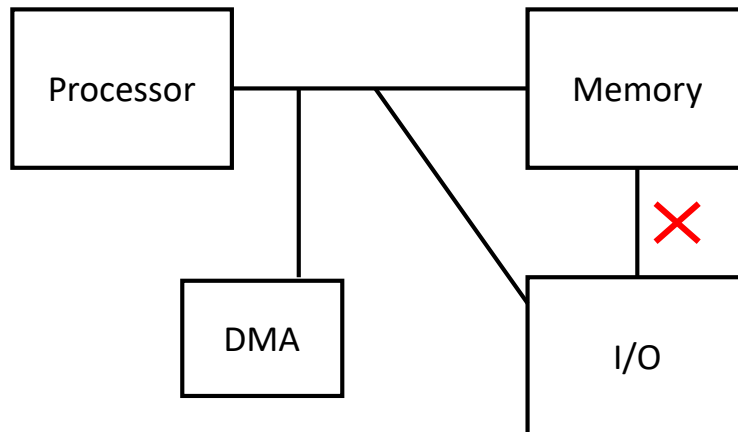
NMI : It is hardware interrupt. Vector for NMI is 0002. processor will execute ISR from this location

INTR : It is interrupt request by device to the processor

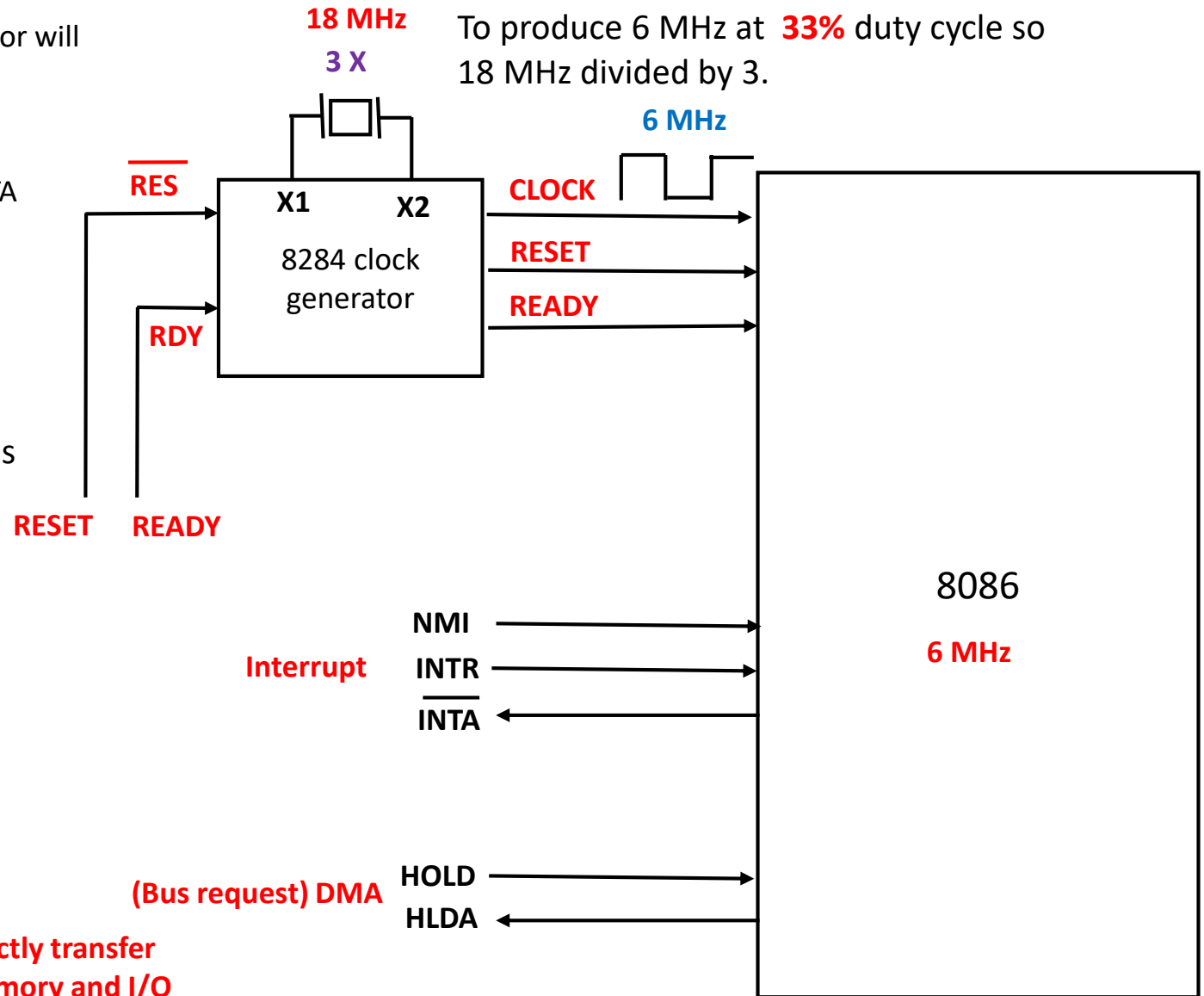
INTA : In response to INTR signal processor will generate 2 INTA pulse During these pulse device will calculate vectored address and pass to the processor and according to calculated vector processor will executes ISR

(Bus request) DMA

- Direct memory access is used to control the system bus
- If DMAC wants to control the system bus then it will send **HOLD** signal to processor
- In response to HOLD signal processor will release the system bus control and generate **HLDA** signal to handover the control of bus system to DMAC

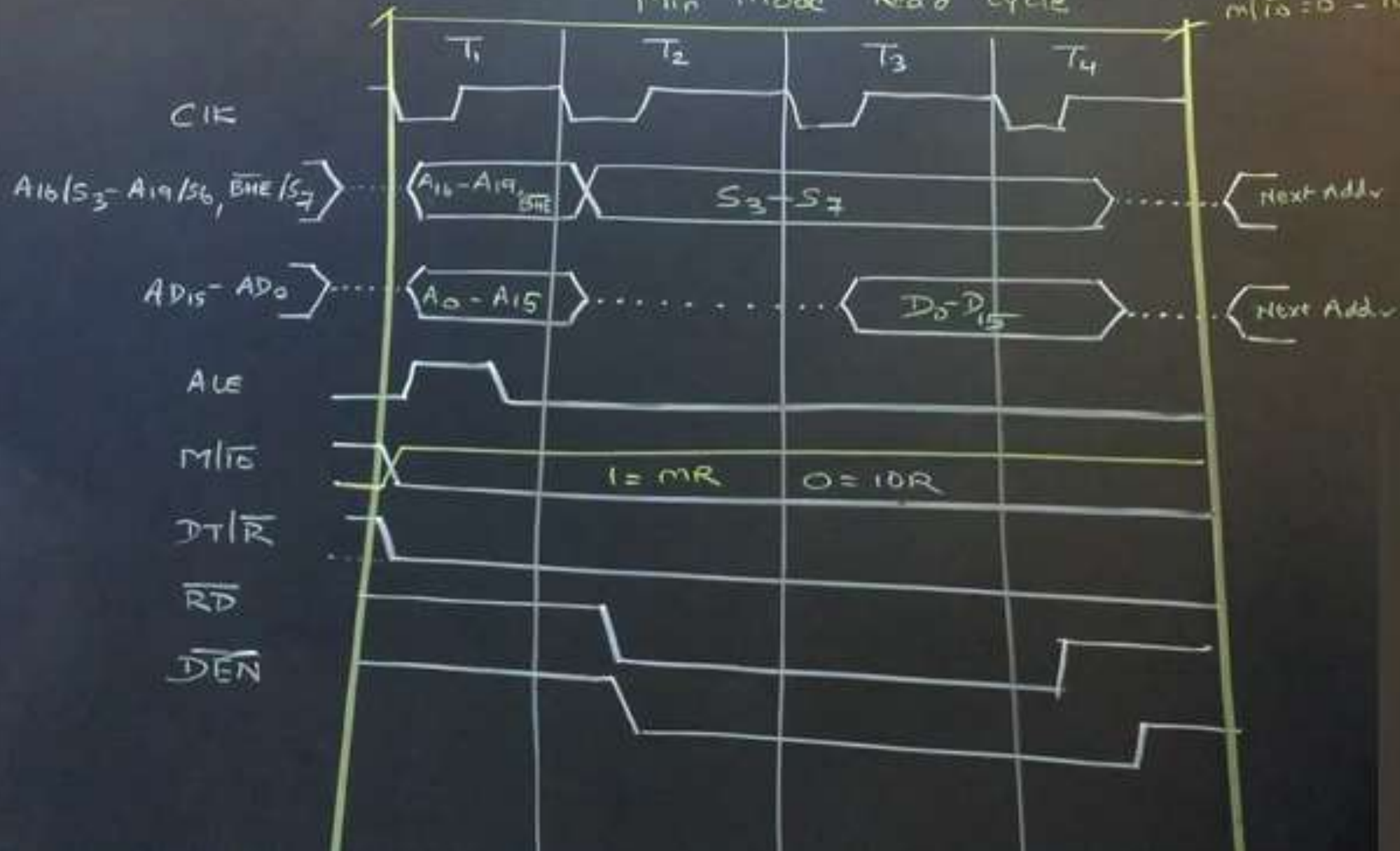


Can not directly transfer data bet memory and I/O for ths required concept DMA



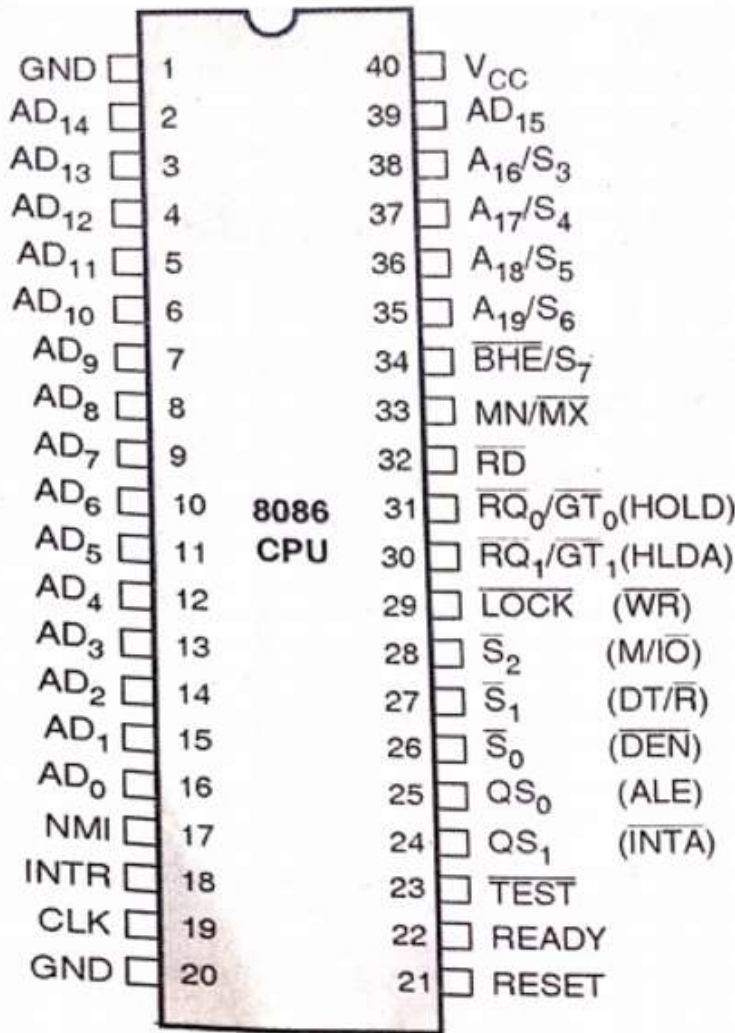
Min Mode Read Cycle

$M/\overline{IO} = 1 - MR$
 $M/\overline{IO} = 0 - IOR$

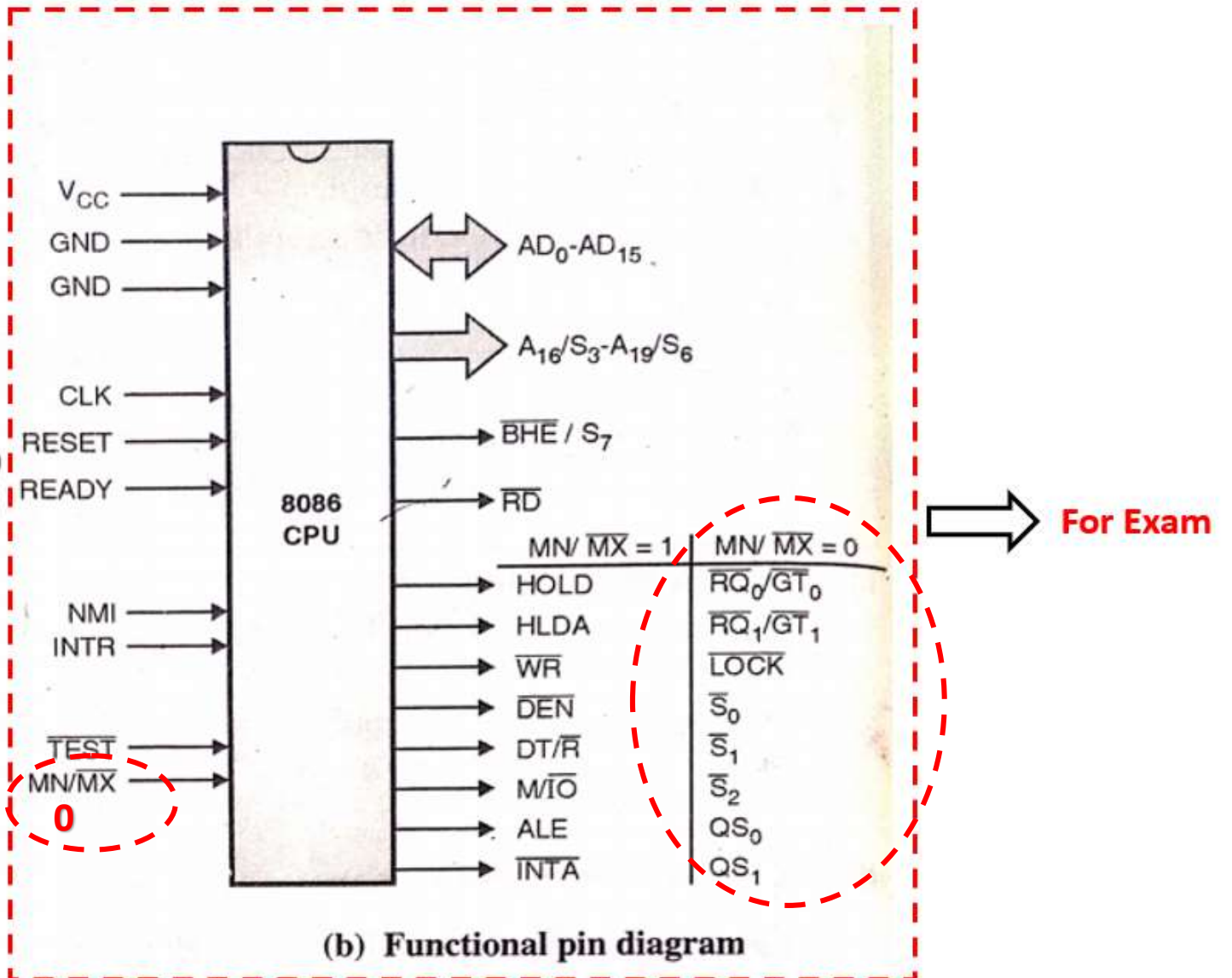


Maximum mode of 8086

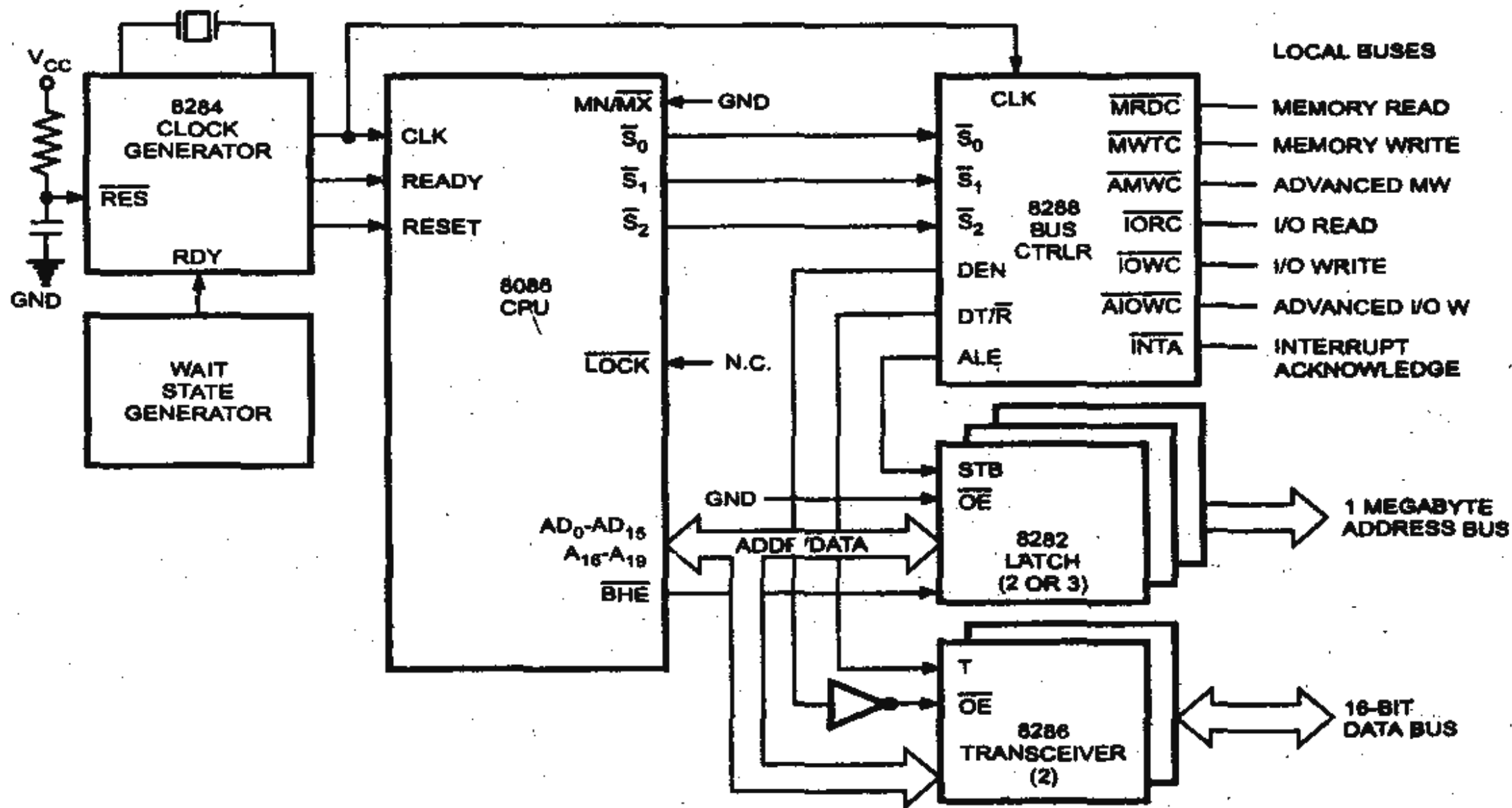
Pin diagram of 8086

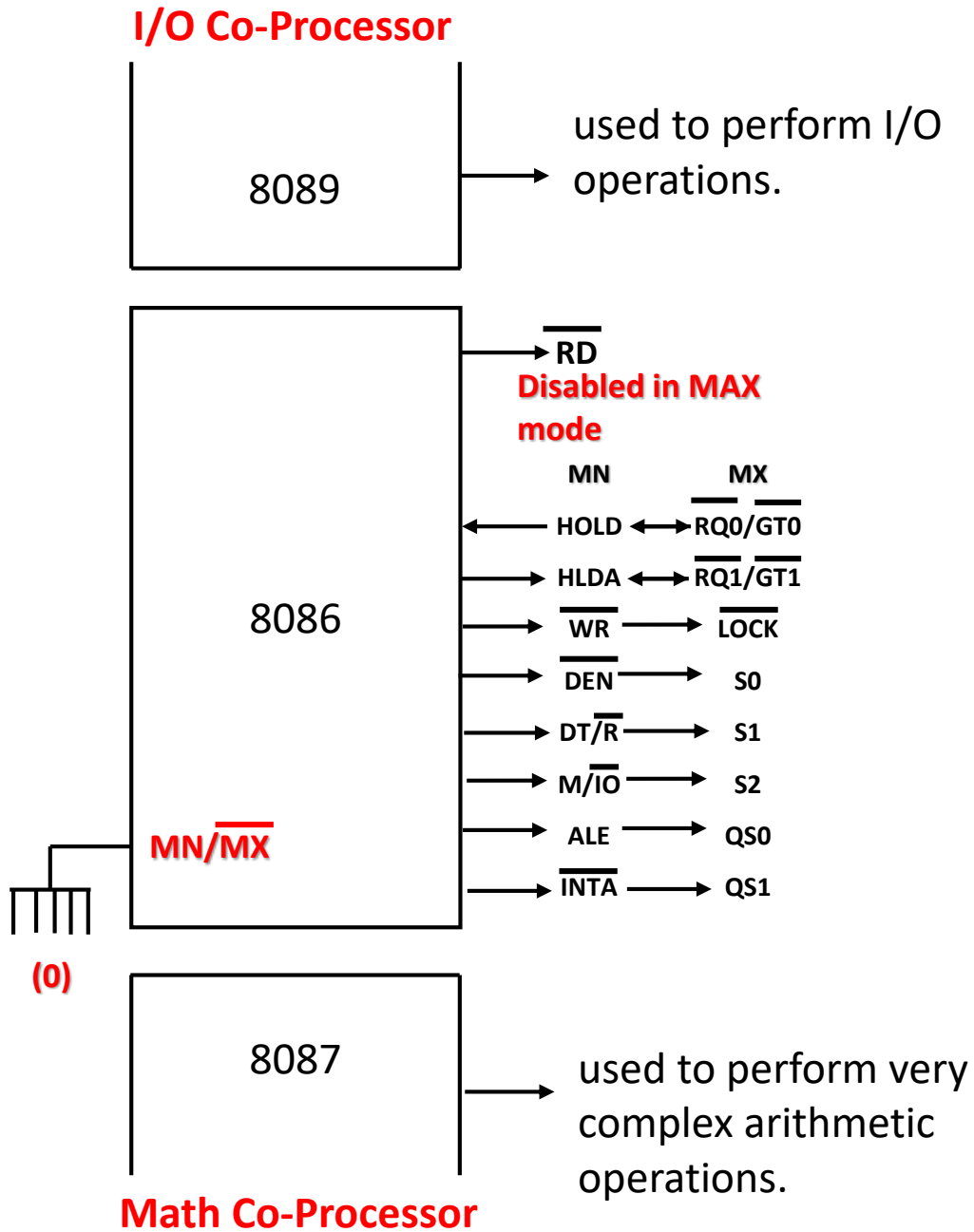


(a) Pin configuration



(b) Functional pin diagram



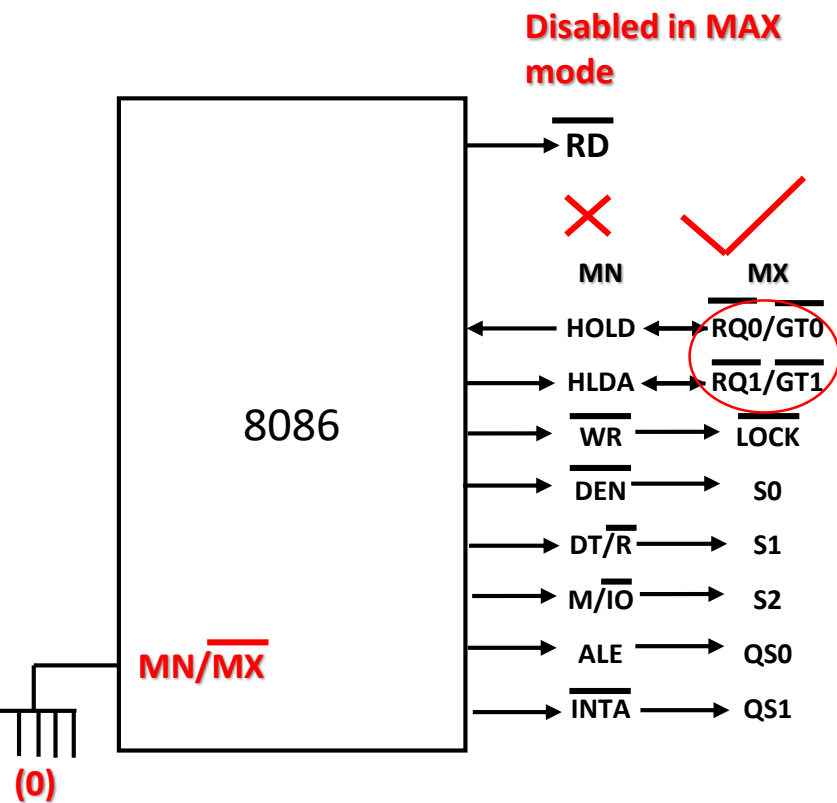


In maximum mode there are multiple processors. We can connect 8087 and 8089 co-processors with 8086.

To perform **multiprocessing** maximum mode is used.



Process simultaneously



RQ – GT Request and Grant :

- The term multiprocessing means more than one processors are working simultaneously.
- Even though there are three processors but there is always only one system bus.
- Other processors like 8087 and 8089 are requesting 8086 for control of system bus.
- The one who control the system bus is called as bus master and by default 8086 is bus master.
- If any one processor wants to become bus master or wants to access system bus it will request to 8086 through **RQ bar** pin.
- Then 8086 will release the system bus and grant the permission through **GT bar** pin.

1. If 8087 wants to become bus master it will send request to 8086 using RQ bar.
2. After receiving RQ bar signal 8086 releases the system bus and grant the permission by sending GT bar signal.
3. Now 8087 is bus master, it will perform its task and after finishing its work it will send release signal to 8086.

This three signals combined in single line.

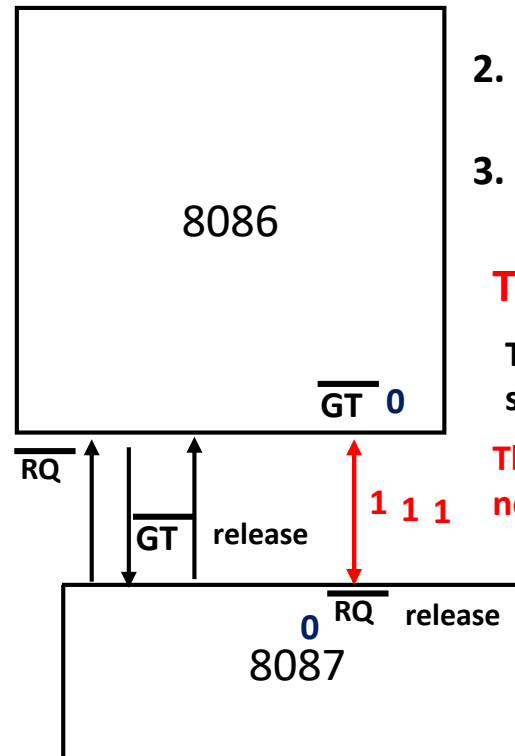
There two co-processor, hence there are two request and grant signals ie. RQ0-GT0 and RQ1-GT1

This are the active low signal which means when there is nothing then status of signal is 1.

When request occurred it will become 0 and immediately gose high 1.

Processor will send 0. RQ0 – GT0 Highest Priority

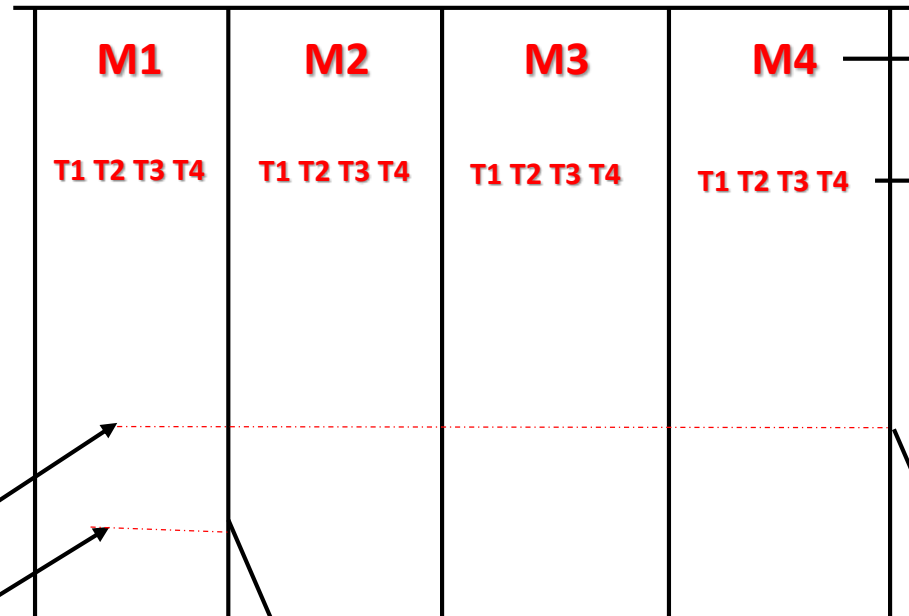
RQ1 – GT1 Lowest Priority



LOCK

Instruction Cycle

Instruction Cycle is the total time required to fetch, decode and execute the instruction.



Machine cycle is memory read/write

T- states required to complete one machine cycle

While processor doing it's task there two events that can disturb the processor.

1. Interrupt Request 2. Bus Request

When interrupt request occurs processor will execute its current instruction and the it will give service.

When interrupt occurs processor has to execute ISR there fore it will first execute current instruction

Service

Service

When bus request occurs, then processor executes its current machine cycle and then provide service.

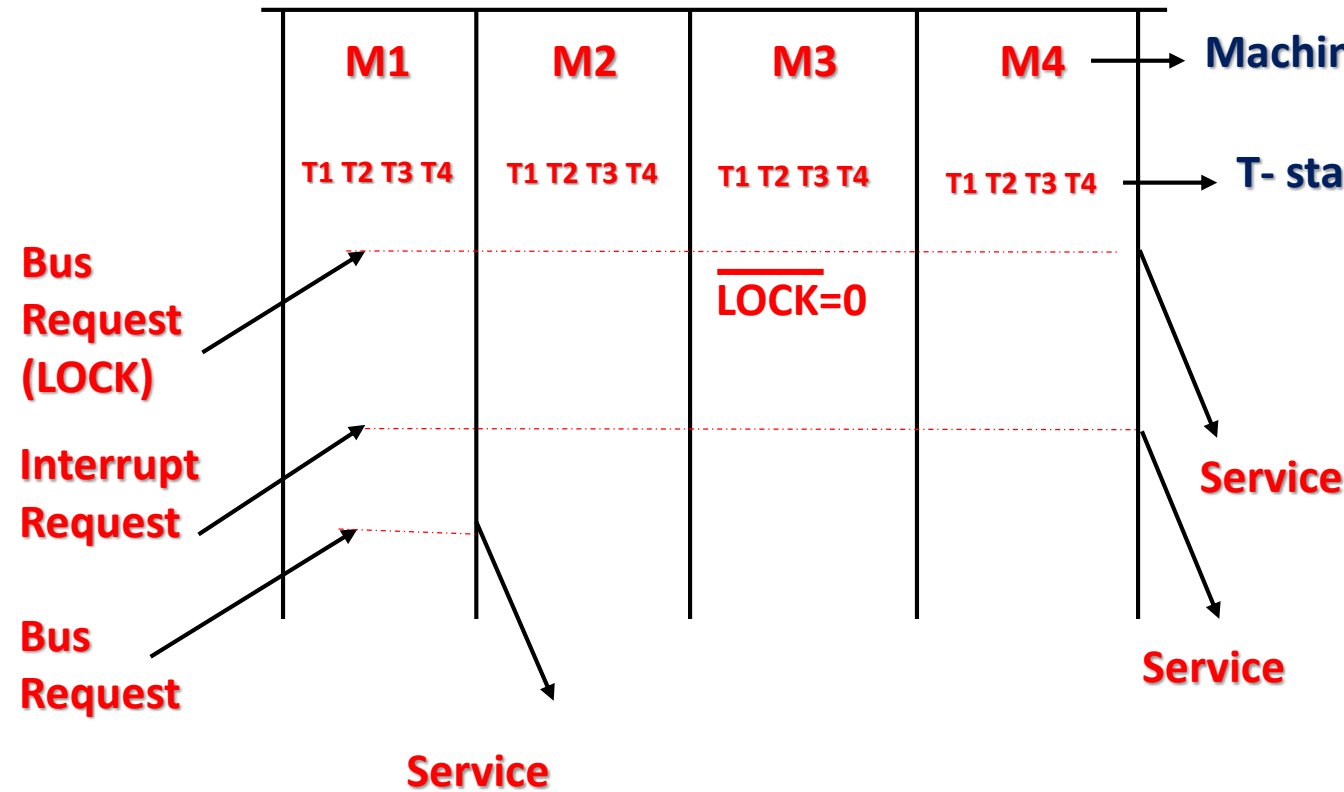
When bus request occurs there is no ISR execution hence processor executes current m/c cycle

processor pause the execution and transfer the bus control but processor can not pause in between T states.

LOCK

Instruction Cycle

Instruction Cycle is the total time required to fetch, decode and execute the instruction.



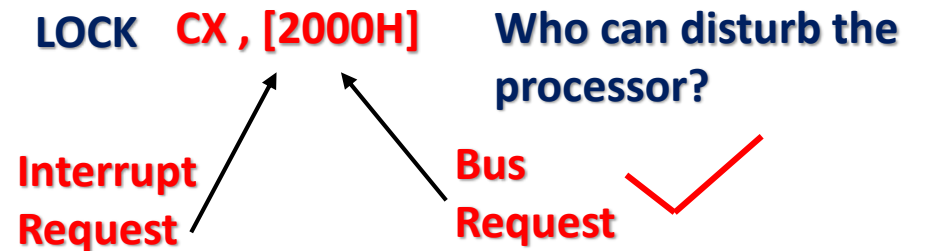
Machine cycle is memory read/write

T- states required to complete one machine cycle

We have write following instruction **CX, [2000H]**

By executing this instruction CX gets content of given memory location.

During this instruction as a programmer we don't want processor to be disturb.



Because of **LOCK** prefix processor execute current instruction and not release the bus control and also inform to all device that bus control is not released by making **LOCK = 0**

Hence assign **LOCK** prefix before the instruction which we don't want to be disturb by bus request.

S0 S1 S2 Status signal :

Control signals are generated by the bus master.

When 8086 is bus master, 8086 is responsible to generate control signals i.e. memory read, memory write

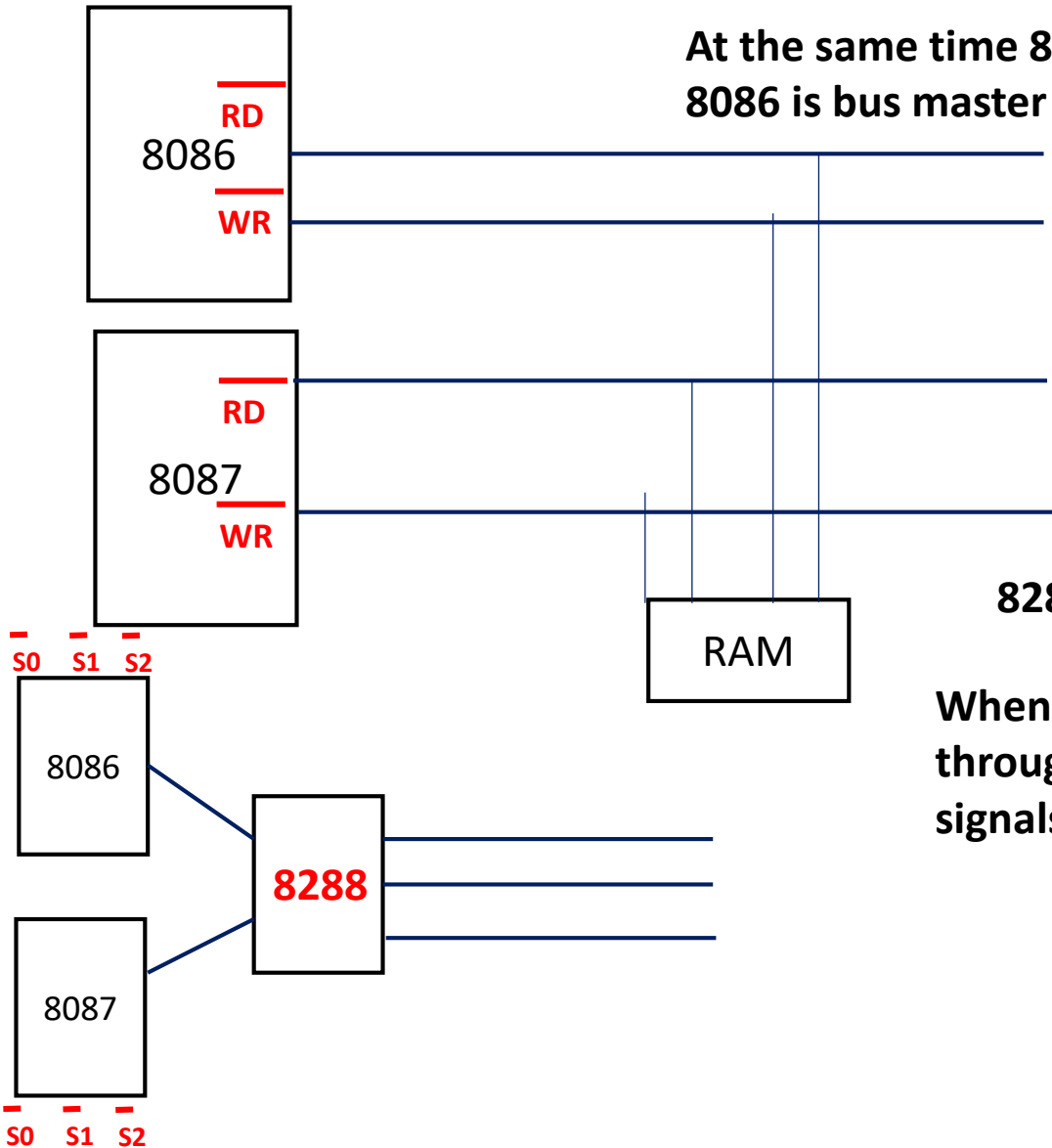
At the same time 8087 can not produce any control signal to access RAM because 8086 is bus master and vice-versa

Hence there is one solution to connect all devices with all processors which is not feasible because it will increase the connectivity and circuitry.

In MAX mode no processor is allow to generate control signals
Who will generate??????

8288 IC is responsible to generate all the control signals

When a processor who is bus master wants to generate control signal through 8288, then the processor has to inform 8288 using status signals.



Who ever is the bus master using status signal will inform 8288 what operation it wants to perform.

8288 will decode the status and produce control signals.

8288 will produce control signals using given table.

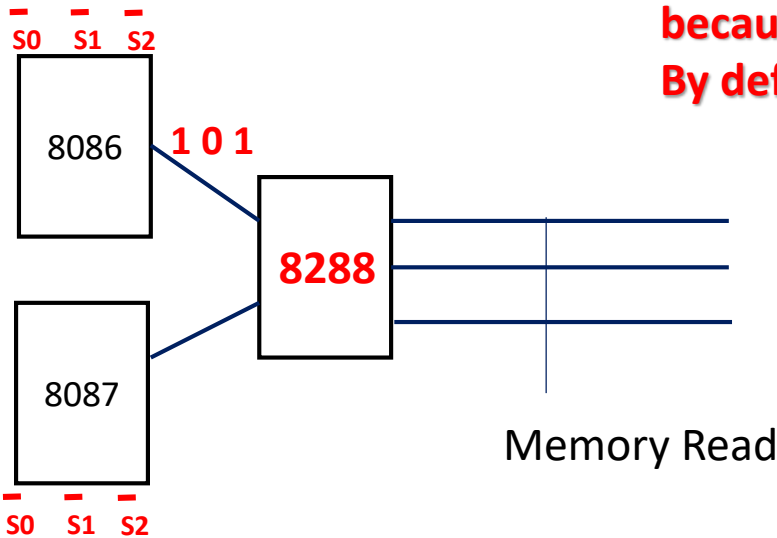
Example

- 1. 8086 wants to produce memory read control signal, then it will generate 1 0 1 status signal and send it to 8288
- 2. 8288 will decode this status signal and produces memory read control signal

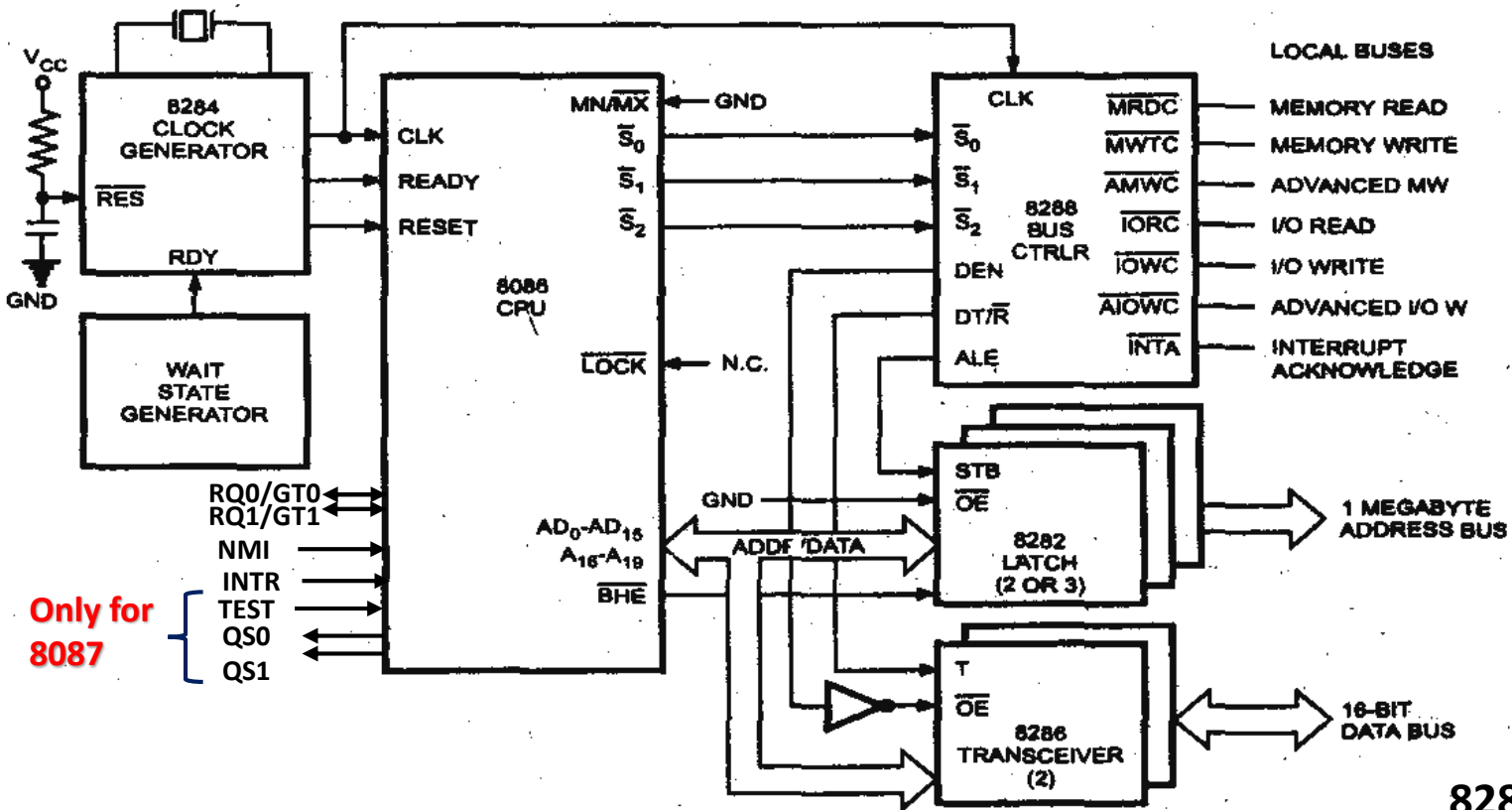
$\overline{S0}$	$\overline{S1}$	$\overline{S2}$	Control Signal
0	0	0	INTA
0	0	1	IO Read
0	1	0	IO Write
0	1	1	Halt
1	0	0	Instruction Fetch
1	0	1	Memory Read
1	1	0	Memory Write
1	1	1	Idle

Both processor can not send same status signal at the same time

because only bus master can generate control signals.
By default 8086 is default bus master so 8086 can send status signal.



When 8087 wants to send status signal first it has to send bus request to 8086 if permission granted then 8087 will become bus master and produces control signals.



- 8284 clock generator is responsible to produce clock signals. These synchronised signals are applied to 8086 and other connected devices.
- There are 21 multiplexed lines.
- When ALE = 1, they carry address & ALE = 0, data.
- These 21 lines are given to 2 – devices, 8282 and 8286.
- 8282 latch will select address & 8286 buffer will select data.
- 8288 will produce control signals.
- In maximum mode all buses (address, data, control) control by 8288.
- In maximum mode 8288 is responsible to generate ALE signal instead of any processor.

8288 required two information :

When to make ALE = 1

Start of new m/c cycle

When to make ALE = 0

Which T-state is going on

Control signal informs about new operation so ALE will be high

Clock signal informs about T-state .

Machine cycle is active for 4T states

ALE is high for only 1st T-state.

- 1) **8086 works in Minimum Mode, when $\overline{MN}/\overline{MX} = 1$.**
- 2) **In Minimum Mode, 8086 is the ONLY processor in the system.**
The Minimum Mode circuit of 8086 is as shown above.
- 3) Clock is provided by the 8284 Clock Generator.
- 4) Address from the address bus is latched into 8282 8-bit latch.
Three such latches are needed, as address bus is 20-bit.
The ALE of 8086 is connected to STB of the latch.
The ALE for this latch is given by 8086 itself.
- 5) The data bus is driven through 8286 8-bit transceiver.
Two such transceivers are needed, as the data bus is 16-bit.
The transceivers are enabled through the $\overline{DEN}$ signal, while the direction of data is controlled by the $\overline{DT}/\overline{R}$ signal. $\overline{DEN}$ is connected to $\overline{OE}$ and $\overline{DT}/\overline{R}$ is connected to T. **Both $\overline{DEN}$ and $\overline{DT}/\overline{R}$ are given by 8086 itself.**

$\overline{DEN}$	$\overline{DT}/\overline{R}$	Action
1	X	Transceiver is disabled
0	0	Receive data
0	1	Transmit data

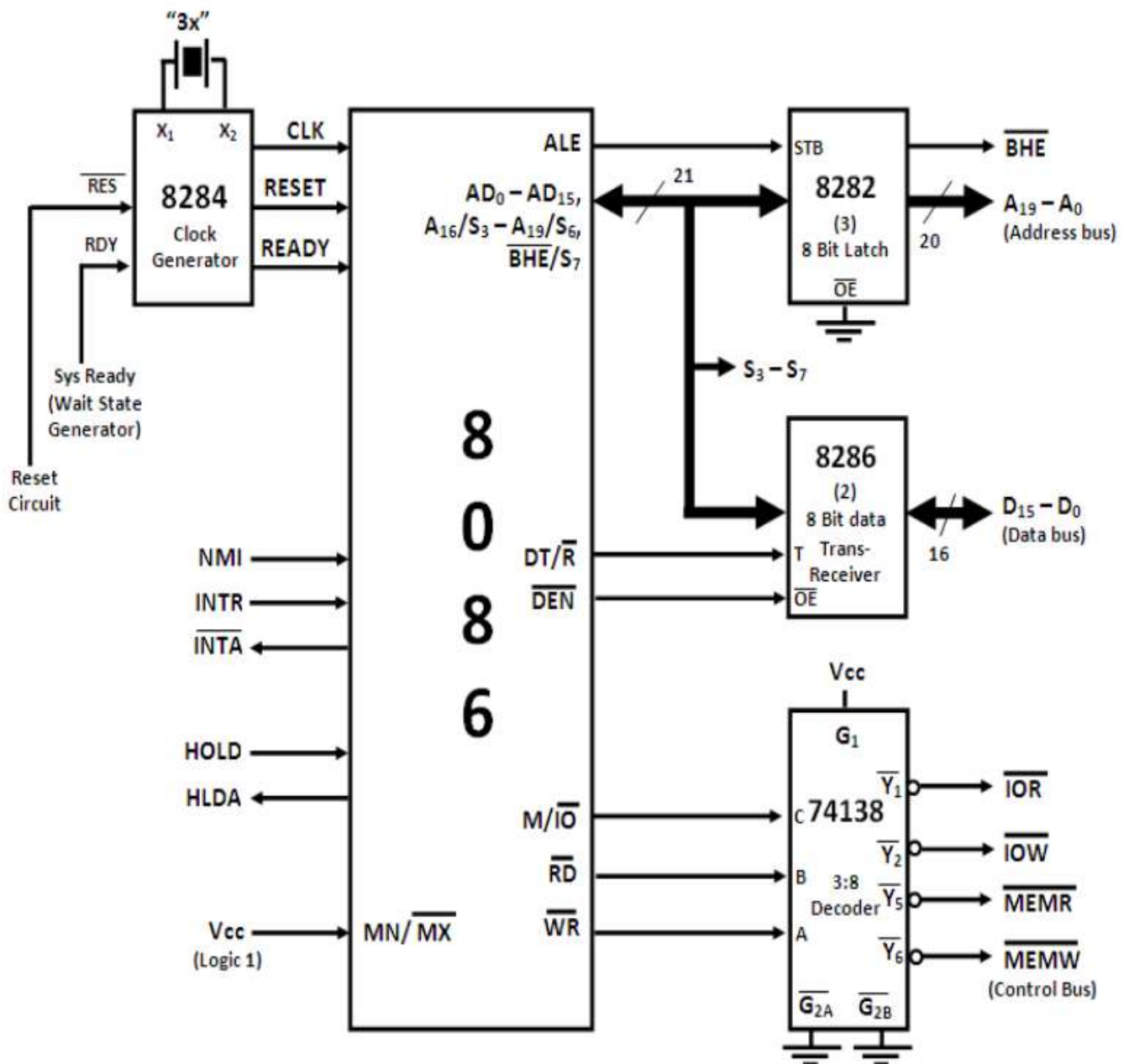
- 6) **Control signals for all operations are generated by decoding $\overline{M}/\overline{IO}$, $\overline{RD}$ and $\overline{WR}$ signals.**

☐ For doubts contact Bharat Sir on 98204 08217

$\overline{M}/\overline{IO}$	$\overline{RD}$	$\overline{WR}$	Action
1	0	1	Memory Read
1	1	0	Memory Write
0	0	1	I/O Read
0	1	0	I/O Write

- 7) **$\overline{M}/\overline{IO}$, $\overline{RD}$, $\overline{WR}$ are decoded by a 3:8 decoder like IC 74138.**
- 8) **Bus Request (DMA) is done using the HOLD and HLDA signals.**
- 9) **$\overline{INTA}$ is given by 8086, in response to an interrupt on INTR line.**
- 10) **The Circuit is simpler than Maximum Mode but does not support multiprocessing.**

8086 MINIMUM MODE CONFIGURATION



1) **8086 works in Maximum Mode, when $\overline{MN}/\overline{MX} = 0$.**

2) **In Maximum Mode, we can connect more processors to 8086 (8087/8089).**

The Maximum Mode circuit of 8086 is as shown above.

3) Clock is provided by the 8284 Clock Generator.

4) The most significant part of the Maximum Mode circuit is the 8288 Bus Controller.

Instead of 8086, the Bus Controller provides the various control signals as explained below.

5) Address from the address bus is latched into 8282 8-bit latch.

Three such latches are needed, as address bus is 20-bit.

This ALE is connected to STB of the latch.

The ALE for this latch is given by 8288 Bus Controller.

6) The data bus is driven through 8286 8-bit transreceiver.

Two such transreceivers are needed, as the data bus is 16-bit.

The transreceivers are enabled through the DEN signal, while the direction of data is controlled by the $\overline{DT}/\overline{R}$ signal.

DEN is connected to $\overline{OE}$ and $\overline{DT}/\overline{R}$ is connected to T.

Both DEN and $\overline{DT}/\overline{R}$ are given by 8288 Bus Controller.

DEN (of 8288)	$\overline{DT}/\overline{R}$	Action
0	X	Transreceiver is disabled
1	0	Receive data
1	1	Transmit data

7) **Control signals for all operations are generated by decoding $\overline{S_2}$, $\overline{S_1}$ and $\overline{S_0}$ signals.** For

$\overline{S_2}$	$\overline{S_1}$	$\overline{S_0}$	Processor State (What the μP wants to do)	8288 Active Output (What Control signal should 8288 generate)
0	0	0	Int. Acknowledge	$\overline{INTA}$
0	0	1	Read I/O Port	$\overline{IORC}$
0	1	0	Write I/O Port	$\overline{IOWC}$ and $\overline{AIOWC}$
0	1	1	Halt	None
1	0	0	Instruction Fetch	$\overline{MRDC}$
1	0	1	Memory Read	$\overline{MRDC}$
1	1	0	Memory Write	$\overline{MWTC}$ and $\overline{AMWTC}$
1	1	1	Inactive	None

8) $\overline{S_2}$, $\overline{S_1}$ and $\overline{S_0}$ are decoded using 8288 bus controller.

9) Bus request is done using $\overline{RQ}$ / $\overline{GT}$ lines interfaced with 8086.

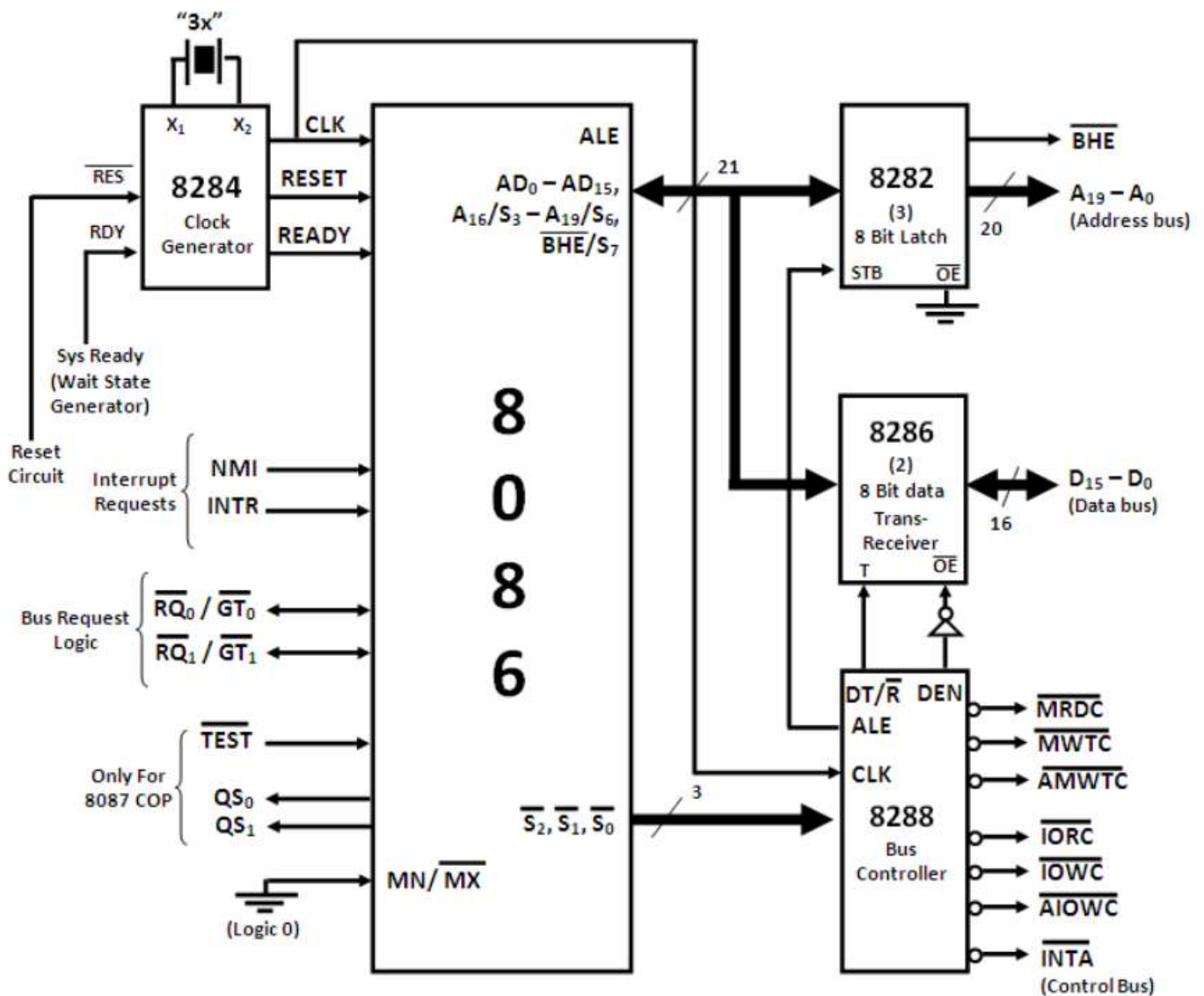
RQ_0/GT_0 has higher priority than RQ_1/GT_1 .

10) $\overline{INTA}$ is given by 8288 Bus Controller, in response to an int. on INTR line of 8086.

11) Max mode circuit is more complex than Min mode but supports multiprocessing hence gives better performance.

12) In max mode, the advanced write signals get activated one T-State in advance as compared to normal write signals. This gives slower devices more time to get ready to accept the data (as μP is writing), and hence reduces the number of "wait states".

8086 MAXIMUM MODE CONFIGURATION



8086 Bus Cycle and Timing Diagram

The BIU initiates all external operations which are also called bus activity. The external bus activities are repetitions of certain basic operations. The basic operations performed by the CPU bus are called bus cycles. Depending on the activities of 8086, the bus cycles can be classified as

- Memory read cycle
- Memory write cycle
- I/O read cycle
- I/O write cycle
- Interrupt acknowledge cycle

The processor takes a definite time to perform a bus cycle. The time taken to perform a bus cycle is specified in terms of term T- states. In 8086 processor the time duration of one T-state is equal to one time period of the interval clock of the processor.

Timing diagram provides information about the various conditions (high state or low

state) of the signals while a bus cycle is executed. **Timing diagram is a graphical representation of the operations of microprocessor with respect to time.**

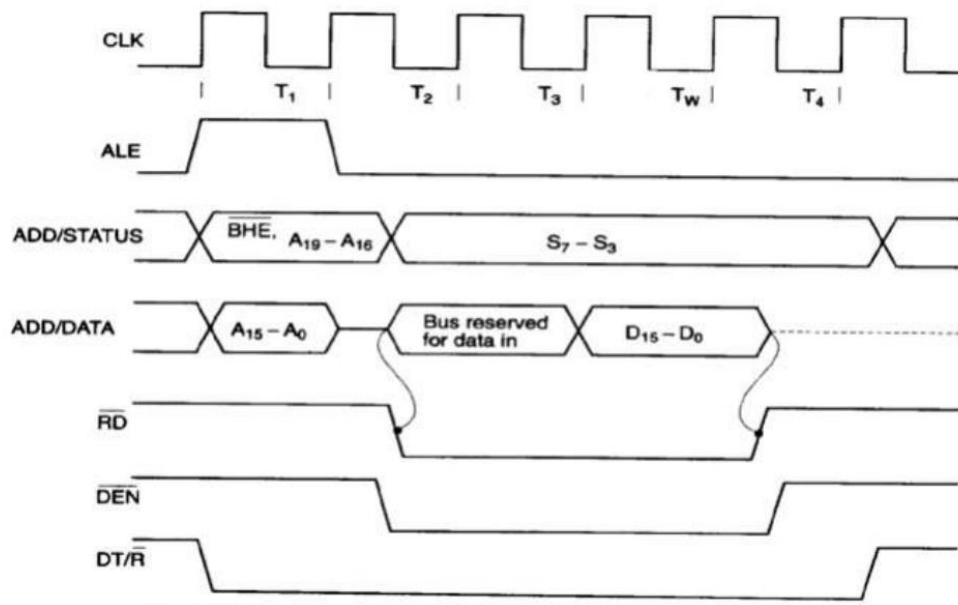
State: one cycle of the clock is called state.

Machine cycle: The basic microprocessor operation such as reading a byte from memory or writing a byte to a port is called machine cycle and made up of more than one state.

Instruction cycle: The time required for a microprocessor to fetch and execute an entire instruction is called Instruction cycle and made up of more than one machine cycle.

An instruction cycle is made up of machine cycles, and a machine cycle is made up of states. The time for a state is determined by the frequency of the clock signal.

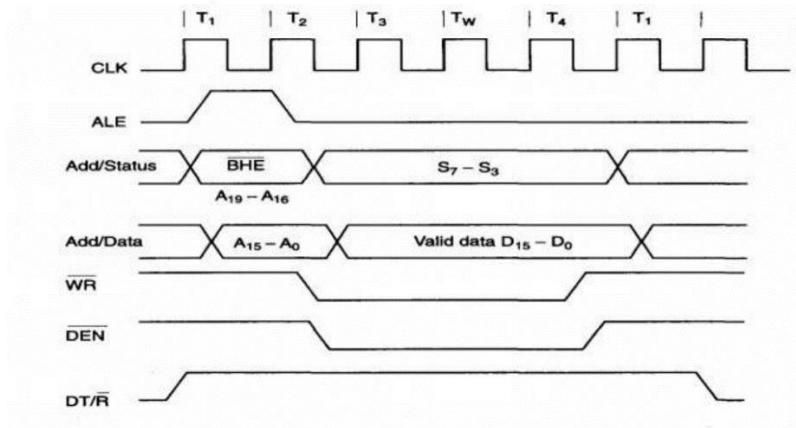
Read cycle timing diagram for Minimum mode



The best way to analyze a timing diagram such as the one to think of time as a vertical line moving from left to right across the diagram.

- The read cycle begins in T1 with the assertion of the address latch enable (ALE) signal and also M/IO' signal.
- During the negative going edge of this signal, the valid address is latched on the local bus. The BHE' and A0 signals address low, high or both bytes.
- From T1 to T4, the M/IO' signal indicate a memory or I/O operation. At T2, the address is removed from the local bus and is sent to the output. The bus is then tristated. The read (RD) control signal is also activated in T2.
- The read (RD) signal causes the addressed device to enable its data bus drivers. After RD goes low, the valid data is available on the data bus. The addressed device will drive the READY line high. When the processor returns the read signal to high level, the addressed device will again tristate its bus drivers

Write cycle timing diagram for Minimum mode



A write cycle also begins with the assertion of ALE and the emission of the address. The M/IO' signal is again asserted to indicate a memory or I/O operation.

- In T₂, after sending the address in T₁, the processor sends the data to be written to the addressed location. The data remains on the bus until the middle of T₄ state. The WR' becomes active at the beginning of T₂ (unlike RD' is somewhat delayed in T₂ to provide time for floating).
- The BHE' and A₀ signals are used to select the proper byte or bytes of memory or I/O word to be read or written.
- The M/IO', RD' and WR' signals indicate the types of data transfer as specified in Table

M/IO	RD	WR	Transfer Type
0	0	1	I/O read
0	1	0	I/O write
1	0	1	Memory read
1	1	0	Memory write

Bus Arbitration / Bus Contention

Multiprocessor system

Closely coupled system

Closely coupled configuration is similar to the coprocessor configuration, i.e. both share the same memory, I/O system bus, control logic, and control generator with the host processor..

Loosely coupled system

Loosely coupled configuration consists of the number of modules of the microprocessor based systems, which are connected through a common system bus. Each module consists of their own clock generator, memory, I/O devices and are connected through a local bus.

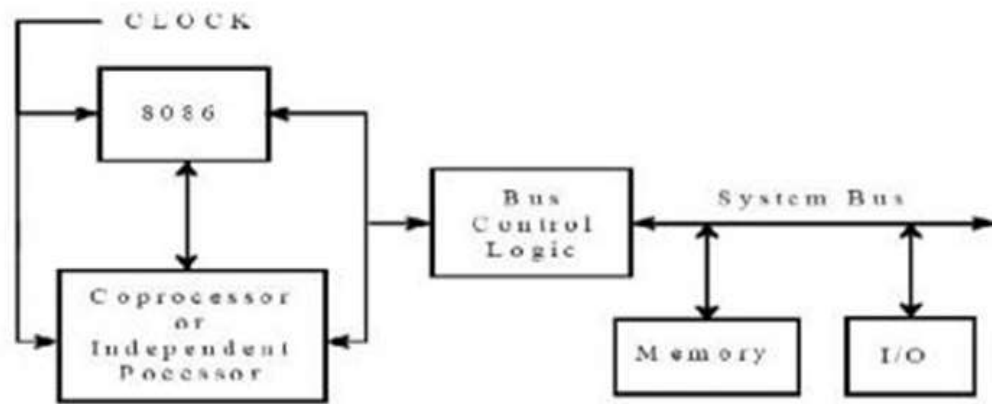
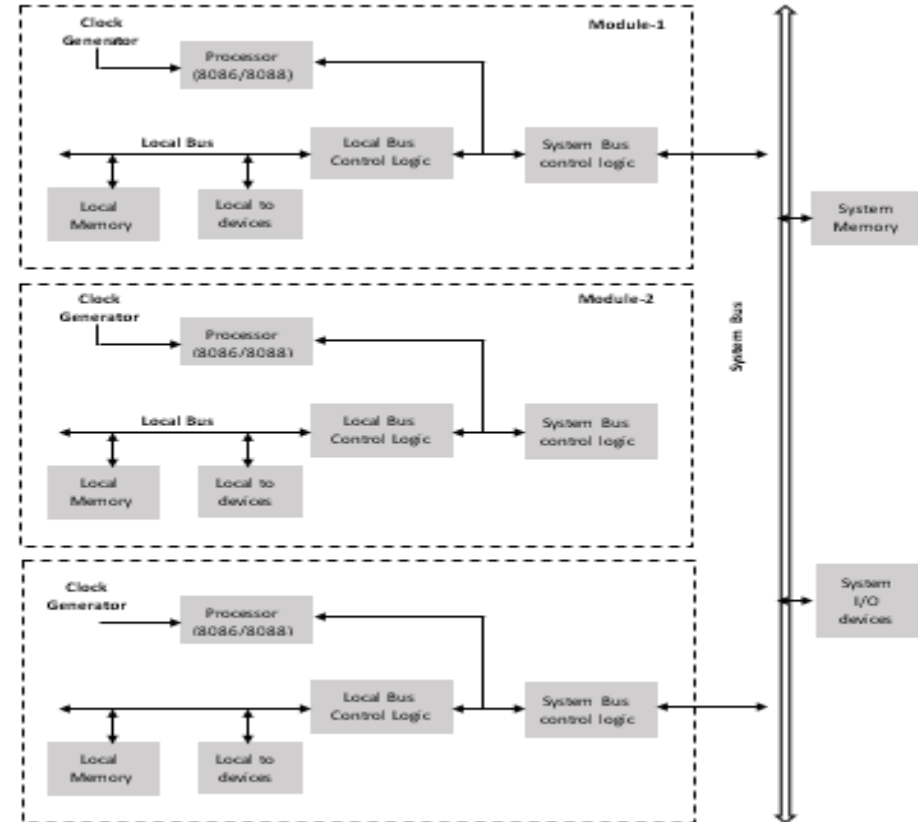


Fig.2.14 closely coupled configuration

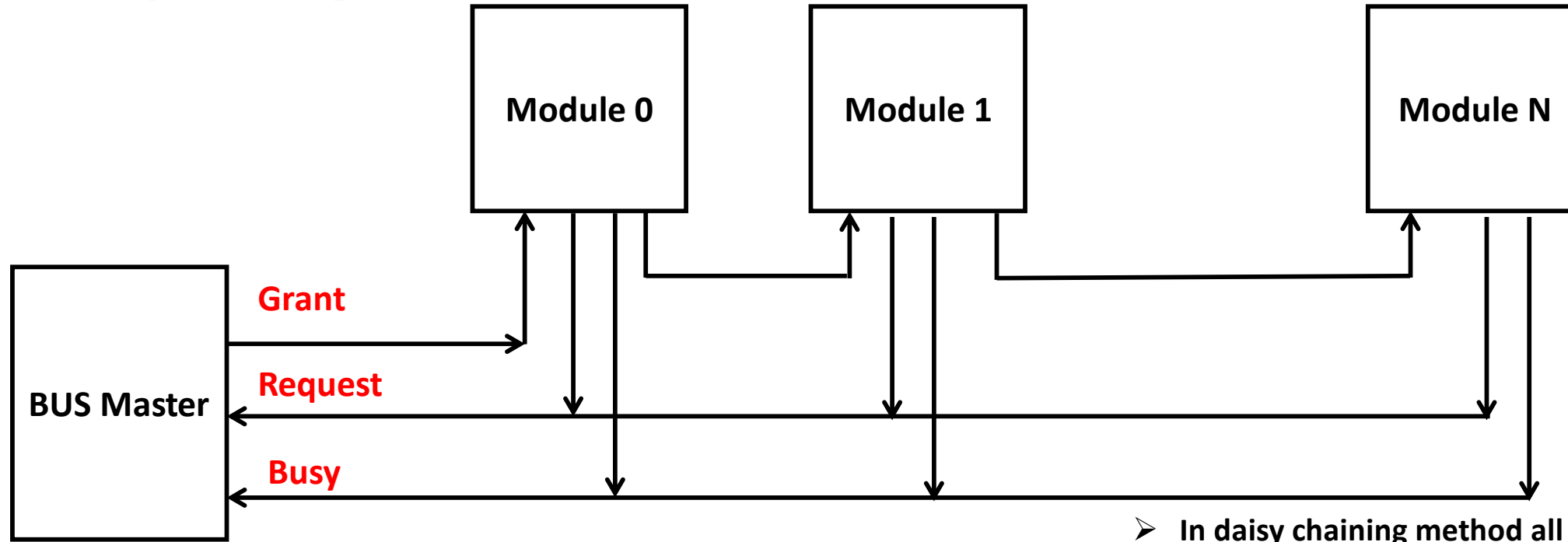


Loosely coupled Multiprocessor

There are universal methods of bus arbitration:

- 1. Daisy chaining method**
- 2. Polling method**
- 3. Independence request method**

1. Daisy chaining method



8259 priority
resolver

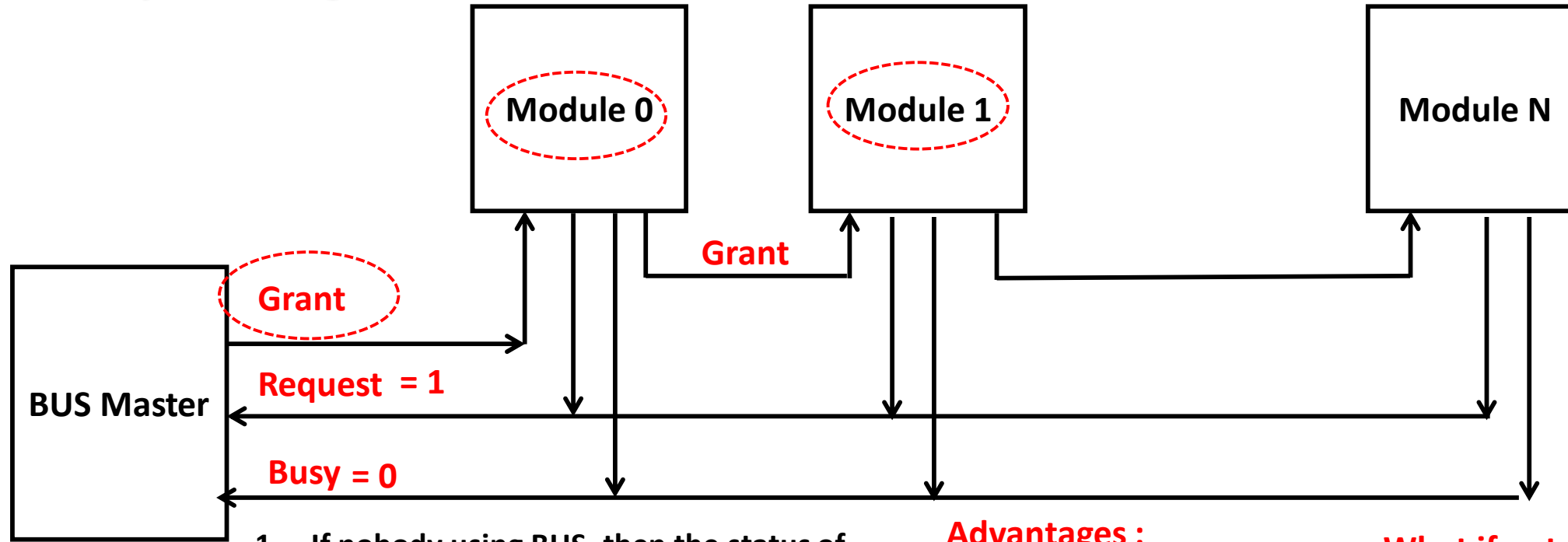
Request : If any module wants bus access then that particular module will send bus request to bus master using this signal.

Grant : If no one module is accessing bus, bus master will provide permission to requested module using grant signal.

Busy : Bus is free or not this is inform by this signal.

- In daisy chaining method all modules are connected with Bus master (8259) who is responsible to provide the system bus access.
- It is also called as priority resolver.
- There are total three important signals. Grant , Request, Busy

1. Daisy chaining method



Every processor is working on its own data

8259 priority resolver

1. If nobody using BUS, then the status of BUSY signal is 0 which means bus is free
2. When request is 1, which means someone wants bus access.

3. If Busy = 0, bus master grant the permission for first module, i.e. physically closest module.

4. If module don't want the bus access it will simply pass the grant to next module

5. If module want's the bus access it will make BUS = 1 and uses the bus.

Advantages :

1. Simplest for small network.

Disdvantages :

1. **Poor performance** : It is slow because one by one grant is passes through modules.

2. **Poor reliability** : If any one module fails grant will be lost.

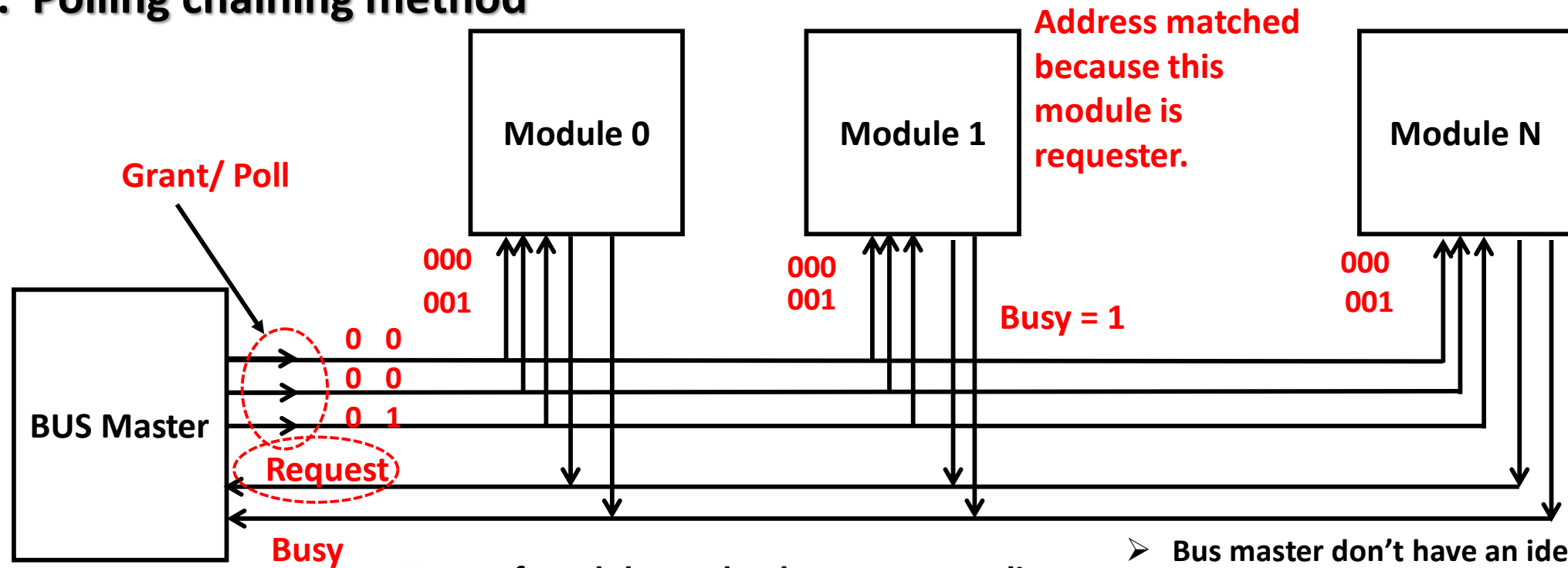
What if network is bigger????

3. **Poor priority mechanism** : get the grant for module which is physically closer the bus arbiter

Example :

If Module 1 and Module 7 request for bus access simultaneously then Module 1 will get first access.

2. Polling chaining method



Now it will send 001 to all

- Advantages :**
- 1. Priority is best
 - 2. Reliability is best
- If any module fails doesn't matter

Disadvantage :

Physical no change. No more module can add due to polling.

N no. of modules and only one request line

1. If there is a bus request i.e. Request = 1, priority resolver not circulate the grant signal. Instead of that it will poll (ask) to every module using poll lines.

- Bus master don't have an idea about the requested module.
- Bus master will ask every module , about the request by polling

Example :

If module 1 wants the bus access then module 1 will send request

Bus master don't know the address of requested module. So it will start with polling

First it will send 000 to all

Address matched with module 0 but he was not the requester & for M1 address not matched

Poll Numbers = Address of every module

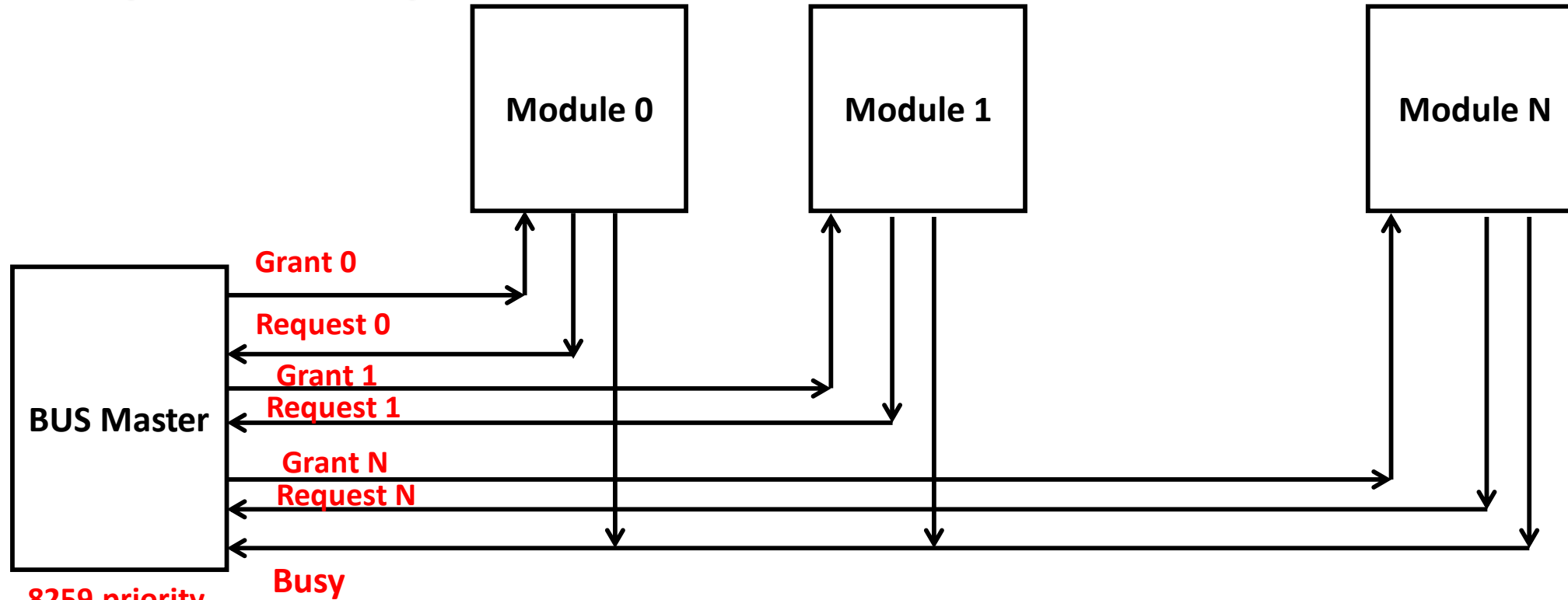
M0	000
M1	001
....	
M7	111

- If 8 modules then,
 $2^3 = 8$ addresses
2. If poll matches, then corresponding module will set the BUSY signal. i.e. **BUSY = 1**

8259 priority resolver

Example :

3. Independence Request method



8259 priority
resolver

In independence request method there
separate grant and request signal for all the
modules and only one Busy signal

Advantages :

- Super in priority and reliability
- Faster

Disadvantages:

- No simple if modules increases.

If any module wants bus access it will send bus
request and if busy = 0 then it will receive
grant signal from bus master.

After receiving grant, corresponding module
set the Busy signal i.e. Busy = 1

BUS CONTENTION / BUS ARBITRATION / PRIORITY RESOLVING SCHEMES

In a Loosely coupled system all processors can use their local bus simultaneously. But the system bus can be used by only one module at a time. Hence there is contest for the system bus. This is called bus contention. It is resolved by various arbitration schemes having different priority methods.

A) Daisy Chain Method

- **All bus masters use the same line for Bus Request.**
- **If the Bus Busy line is inactive, the Bus Controller gives the Bus Grant signal.**
- **Bus Grant signal is propagated serially** through all masters **starting from nearest one.**
- The bus **master**, which requires the system bus, **stops this signal, activates the Bus Busy line and takes control** of the system bus.

Advantage:

- i. **Design is simple.**
- ii. The **number of control lines is less.** Also **adding** new bus masters is **easy.**

Disadvantage:

- i. **Priority** of bus masters is **rigid** and **depends** on the **physical proximity** of the bus masters with the bus arbiter i.e. The one nearest to the Bus Arbiter gets highest priority.
- ii. Bus is granted **serially** and hence a **propagation delay** is induced in the circuit.
- iii. **Failure of one** of the devices may **fail** the entire **system.**

B) Polling Method

- Here also **all bus masters** use the **same** line for **Bus Request.**
- Here the **controller generates** binary **address** for the master.
Eg: To connect 8 bus masters we need 3 address lines ($2^3 = 8$).
- In **response to a Bus Request**, the **controller "polls"** the bus masters by **sending a sequence** of bus master **addresses** on the address lines. Eg: 000,010,100,011 etc
- The selected **master activates** the **Bus Busy** line and **takes control** of the bus.

Advantage:

- i. The **Priority** is **flexible** and can easily be **changed** by **altering** the **polling sequence.**
- ii. If **one module fails**, the entire **system does not fail.**

Disadvantage:

- i. **Adding** more bus masters is **difficult** as **increases** the number of **address lines** of the circuit. Eg: In the above circuit to add the 9th Bus Master we need 4 address lines.

C) Independent Request Method

- Here, **all bus masters** have their **individual Bus Request** and **Bus Grant lines.**
- The **controller thus knows which master has requested**, so bus is granted to that master.
- **Priorities** of the masters are **predefined** so on **simultaneous Bus Requests**, the bus is **granted based** on the **priority**, provided the Bus Busy line is not active.
- The **Controller** consists of **encoder** and **decoder** logic for the priorities.

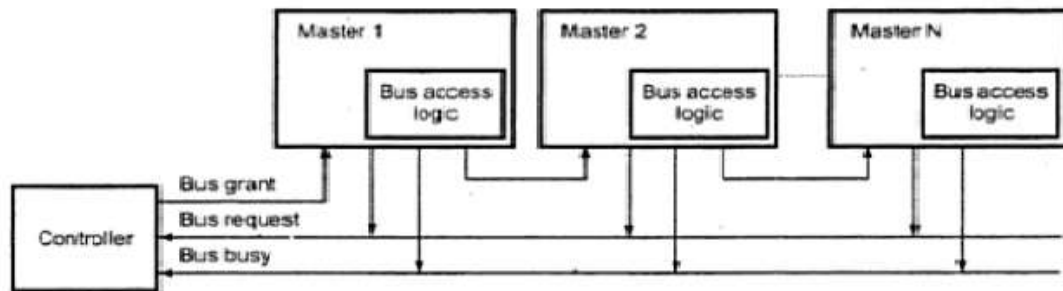
Advantage:

- i. The **Bus Arbitration is fast.**
- ii. The **speed of Bus Arbitration** is **independent** of the **number** of devices connected.

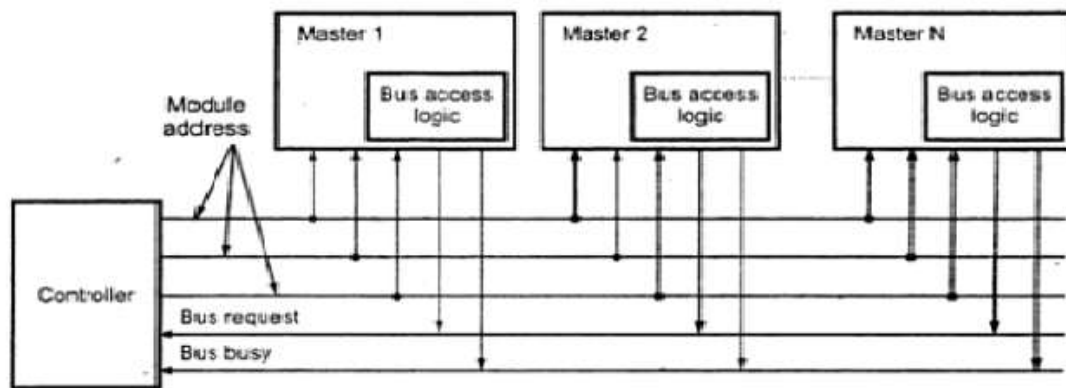
Disadvantage:

- i. The **number of control lines required is more** ($2n$ line required for n devices).

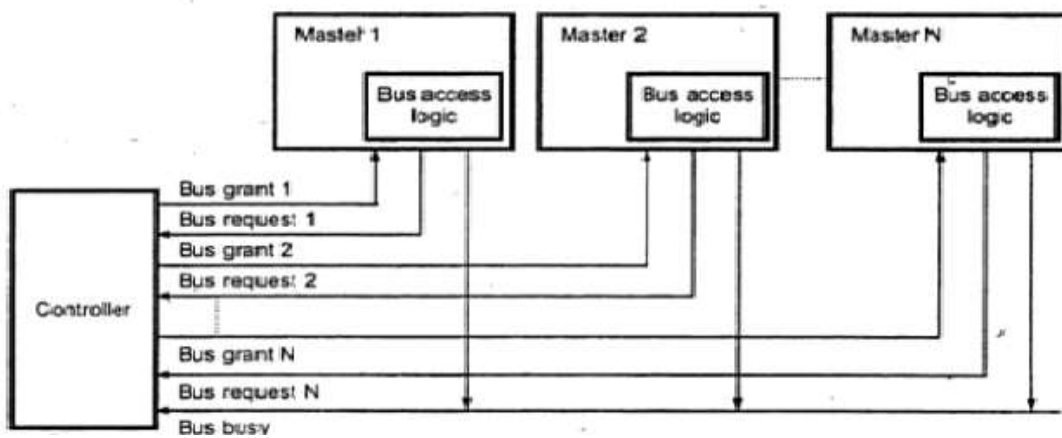
Daisy Chaining



Polling



Independent Requests



Math Co-Processor 8087

Why 8087 is used with 8086/ why 8087 interfaced with 8086??????

1. It is numerical data processor which means processor is design to perform **complex arithmetic** operations.

8086 is also worked on arithmetic operation but they are **very simple**.



Addition, Subtraction, Division, Multiplication

8087 is worked on Real Complex Arithmetic operations like :



Square root, Trigonometry, Laws, X^Y

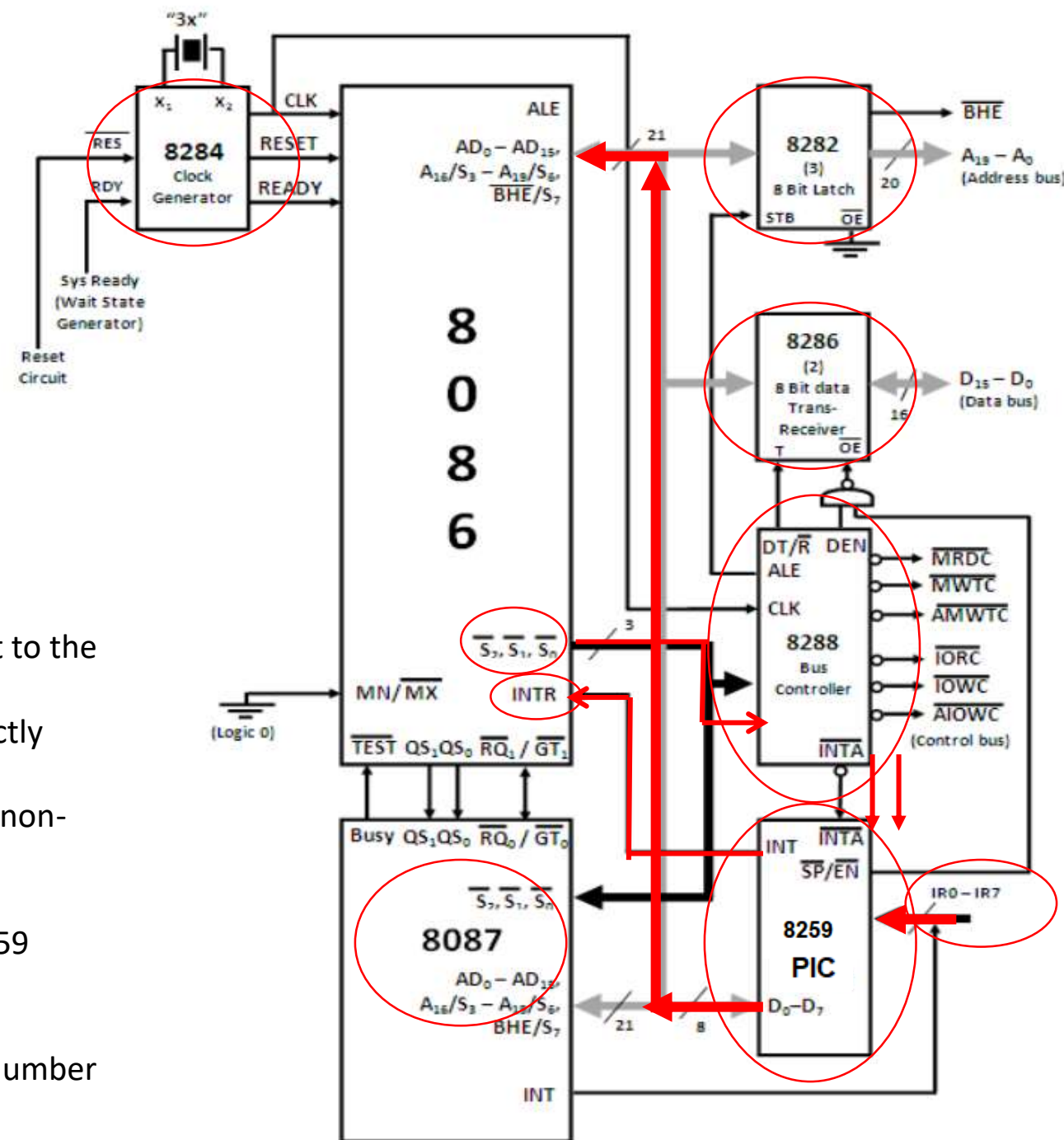
2. 8087 works on very big number i.e. **80 bit number**.
3. 8087 has very **powerful instruction set**. It has **direct instruction** to do practically everything that you see in **scientific calculator**. **Instruction available** for every **button** of **scientific calculator**.
4. 8087 has capability to work with **floating point numbers**. i.e. **fraction**. Example : **square root of (5.7)**

Co-Processor Configuration 8086 - 8087

In the diagram following blocks are there:

1. **8284 Clock generator** which is used to provide clock related signals to 8086 and other connected devices.
2. **8282 latch** which is used separate the address bus.
3. **8286 transceiver** which is used to separate the data bus.
4. **8288 bus controller** which is used to handle the control bus.
5. **8087 math co-processor** to perform complex arithmetic operations.
6. **8259 programmable interrupt controller** to handle the interrupt request.

- **8259** is used to increase the **number of interrupts**.
- 8259 device is used to handle the interrupt requests and send it to the processor.
- Any device which wants to interrupt the processor can not directly interrupt the processor it will send interrupt request to 8259.
- 8259 will send that interrupt request through INTR pin which is non-vectored (Processor don't know the address of an ISR).
- 8086 sends the status signal to 8288 to generate INTA bar s/g
- 8288 will generate the INTA bar signal to ask vector number. 8259 calculates the vector address.
- Again 8086 generate second time INTA bar signal through 8288.
- After receiving second time INTA bar 8259 will provide vector number of an ISR to 8086 and 8086 executes that interrupt.



Who wants to interrupt the 8086 processor?
8087

When 8087 wants to interrupt the processor?

8087 wants to become bus master 

When any device wants **service** or if there is **problem** it will send **interrupt request**

If 8087 wants to become bus master it will send **bus request**

There is a difference between **bus request** and **interrupt request**

- 8087 dose the **floating point** operations.
- During those operations there **6** types of **errors or exceptions** can occurs :
- Precision, divide zero, overflow, underflow, invalid operation, denormal
- If there is any type of error occurred 8087 interrupt the processor
- But it can not directly interrupt to processor.
- It will interrupt through 8259.

8087 is also a processor, it can also become a bus master to perform operations.

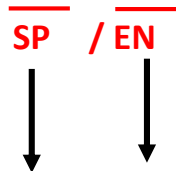
To perform operations it also required clock pulses.

When you reset computer both processor should reset.

When co-processor dealing with other devices ready s/g is used.

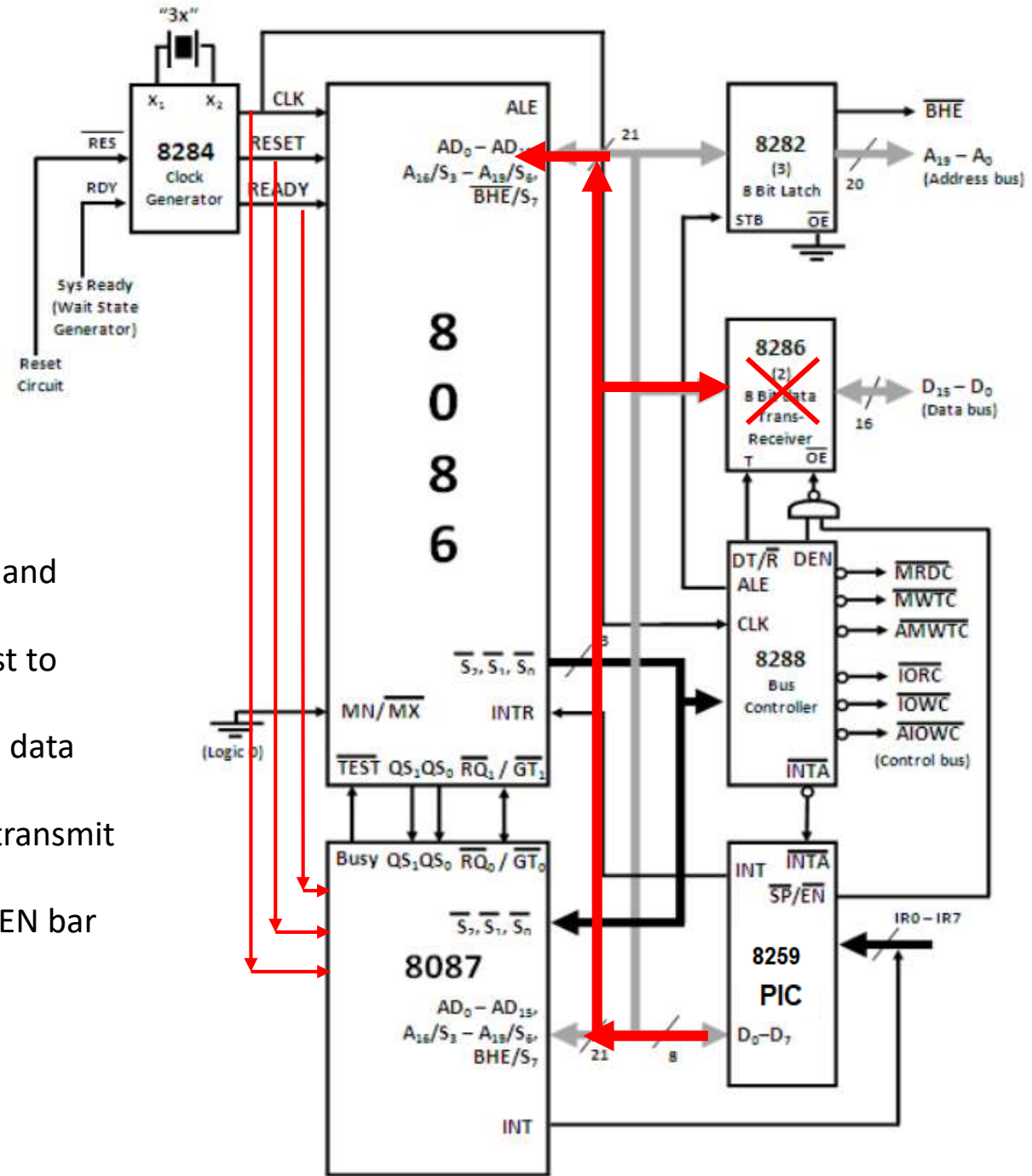
Hence all clock signals are applied to 8087 also

- 8086 or 8087 who ever is a bus master will generate multiplexed bus and 8282 and 8286 is responsible to separate the address and data bus.
- 8087 interrupt to 8086 through 8259. 8259 sends an interrupt request to 8086 and receive 2 – INTA bar signals through 8288.
- On second INTA bar signal 8086 send vector number to 8087 through data bus.
- There is a chance that vector number is mixed up with data which is transmit or receive by 8286. because vector no. can also received by 8286.
- Hence it is responsibility of 8259 to disable the 8286 by using SP bar/EN bar pin



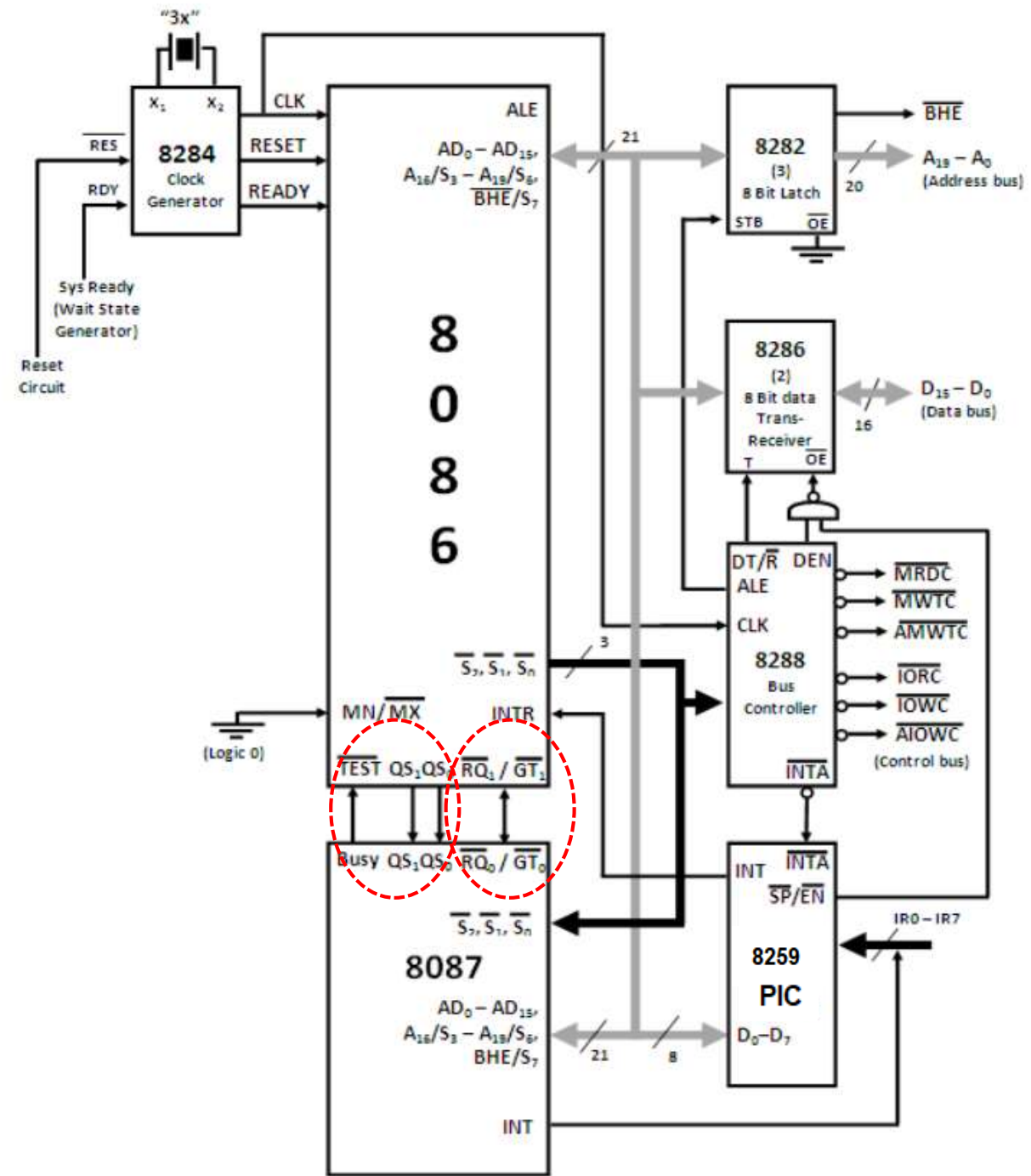
Cascaded mode

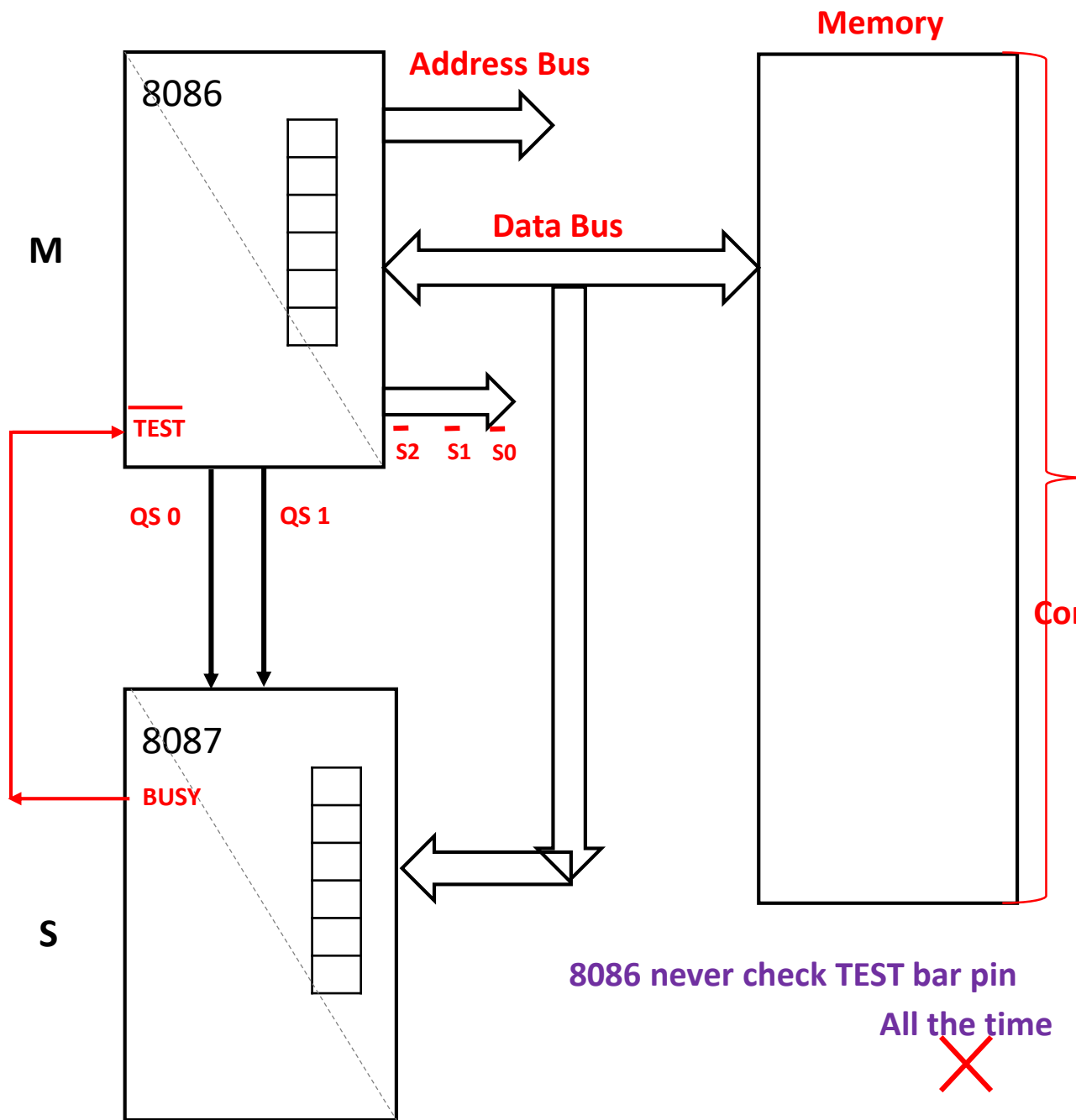
Single mode



$\overline{RQ}$ - $\overline{GT}$

1. If 8087 wants to become bus master it will send request to 8086 using $\overline{RQ}$ bar.
2. After receiving $\overline{RQ}$ bar signal 8086 releases the system bus and grant the permission by sending $\overline{GT}$ bar signal.
3. Now 8087 is bus master, it will perform its task and after finishing its work it will send release signal to 8086.





- 8086 and 8087 both processors can execute the programs.
- 8086 is a Master (M) and 8087 is a slave (S) Processor. Which means **every activity** performed by 8087 is **controlled or given** by 8086.
- When 8086 wants to give new operation to 8087, first 8086 has to know whether 8087 is busy or free.
- Busy means to perform its own operations (complex). If 8087 is busy then it will send 1 on BUSY line which is connected with TEST bar pin of 8086.
- So whenever 8086 wants to give new task to 8087 it will check the status of TEST bar pin.
- If test bar not checked by processor it will send new task and 8087 will leave the current task and start with new task.
- Which means 8086 has to check TEST bar and if it is 1 the 8086 will enter into WAIT state.

Combine Program

- If TEST bar = 0, 8087 not busy & if TEST bar = 1, 8087 busy, 8086 in wait state.
- Which means at time only one processor is working.

Multiprocessing Both processors work together.

Can not achieve multiprocessing

If 8086 checks the TEST bar and TEST bar is 1 **then only** 8086 has to enter into wait state.

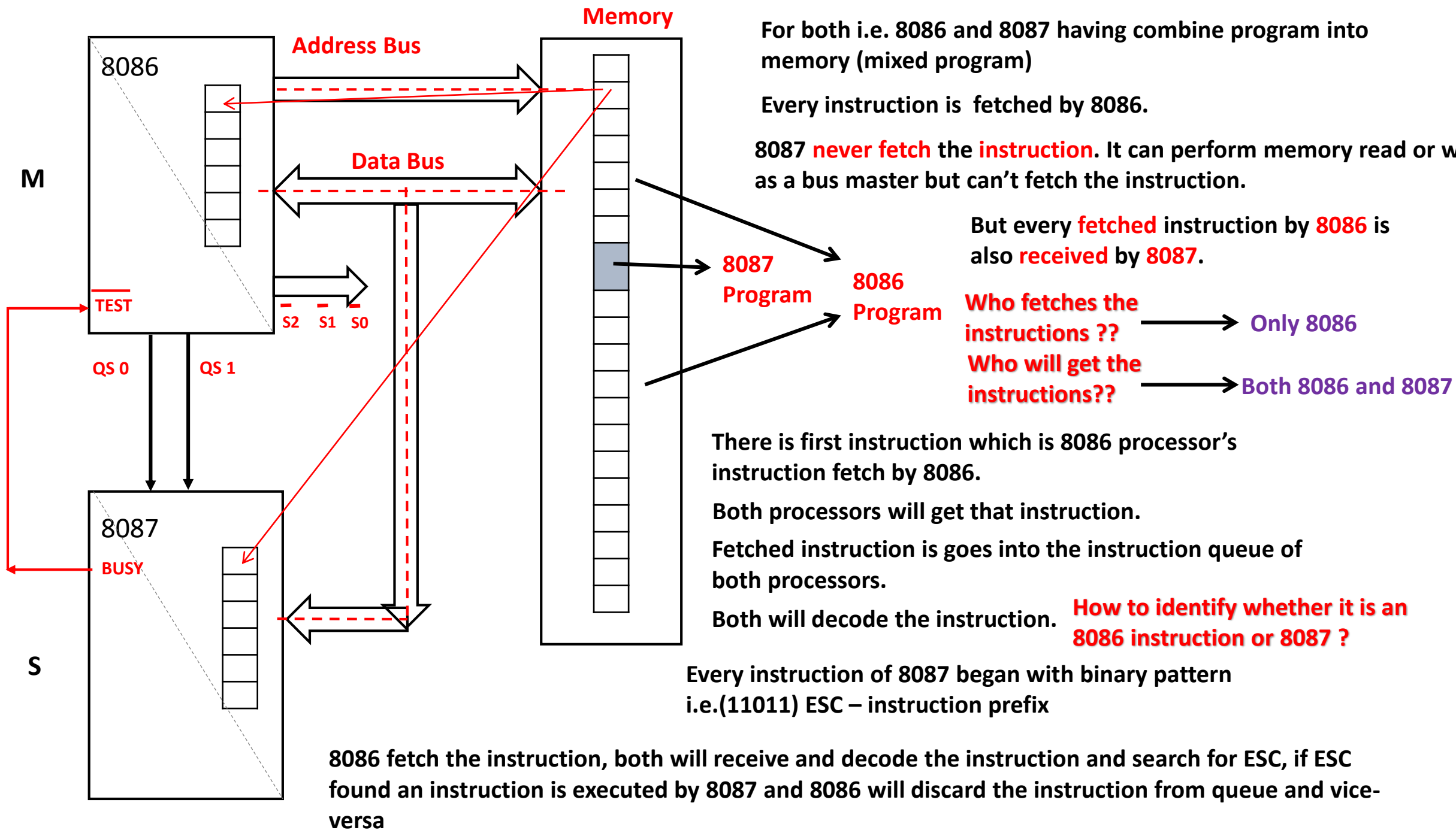
All the time

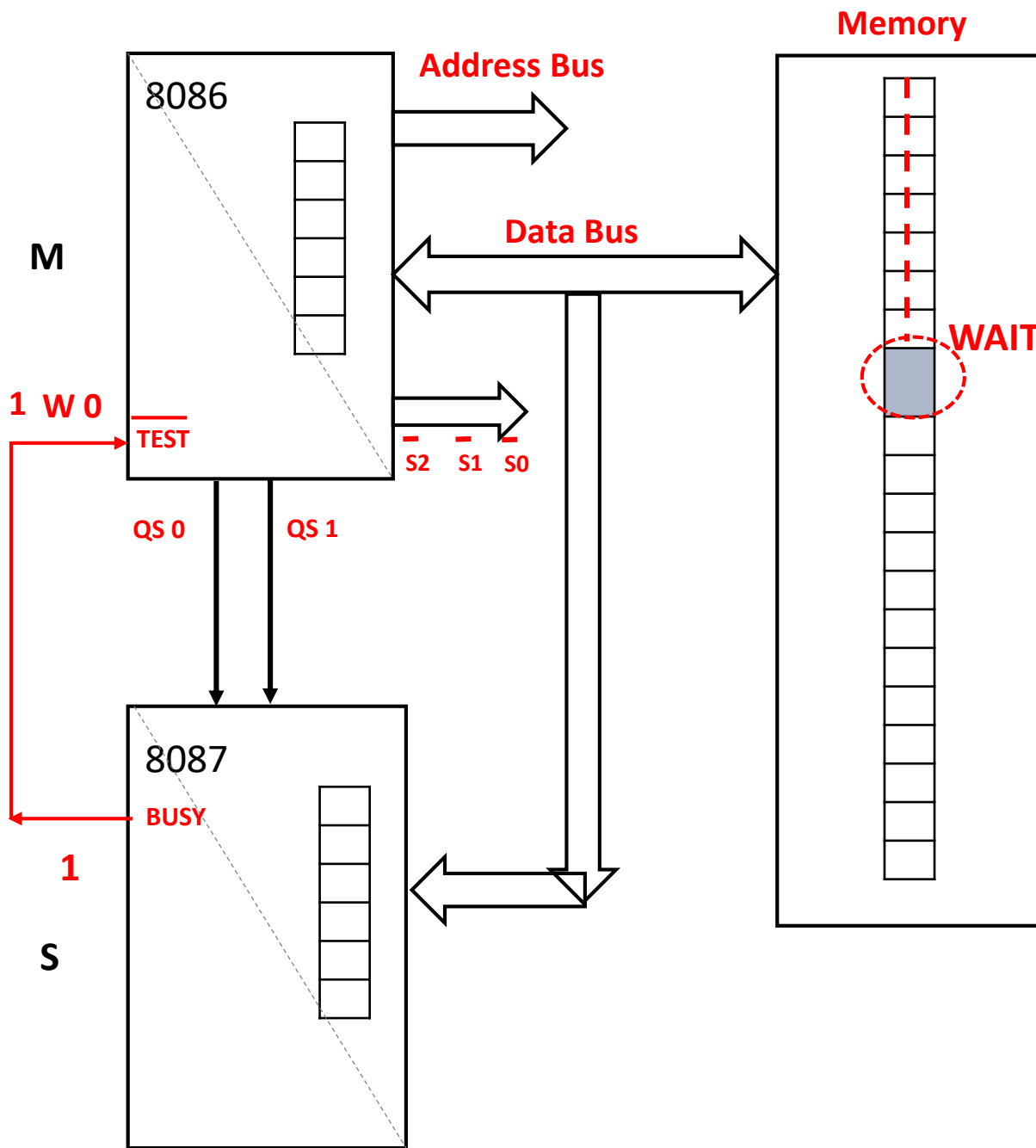


← **When 8086 checks the TEST bar??**

As a programmer we force 8086 to check TEST bar pin.

When should we make the 8086 to check the TEST bar??





As a programmer we force 8086 to check TEST bar pin.

When should we make the 8086 to checks the TEST bar??

All these are the 8086 instructions so no need to check test bar pin.

As a programmer we know that this one is the 8087 instruction.

If this instruction fetch by 8086 it will receive and decode by both and 8087 executes this instruction and leaves the previous instruction.

Hence before fetching this instruction 8086 has to check whether 8087 is free or not

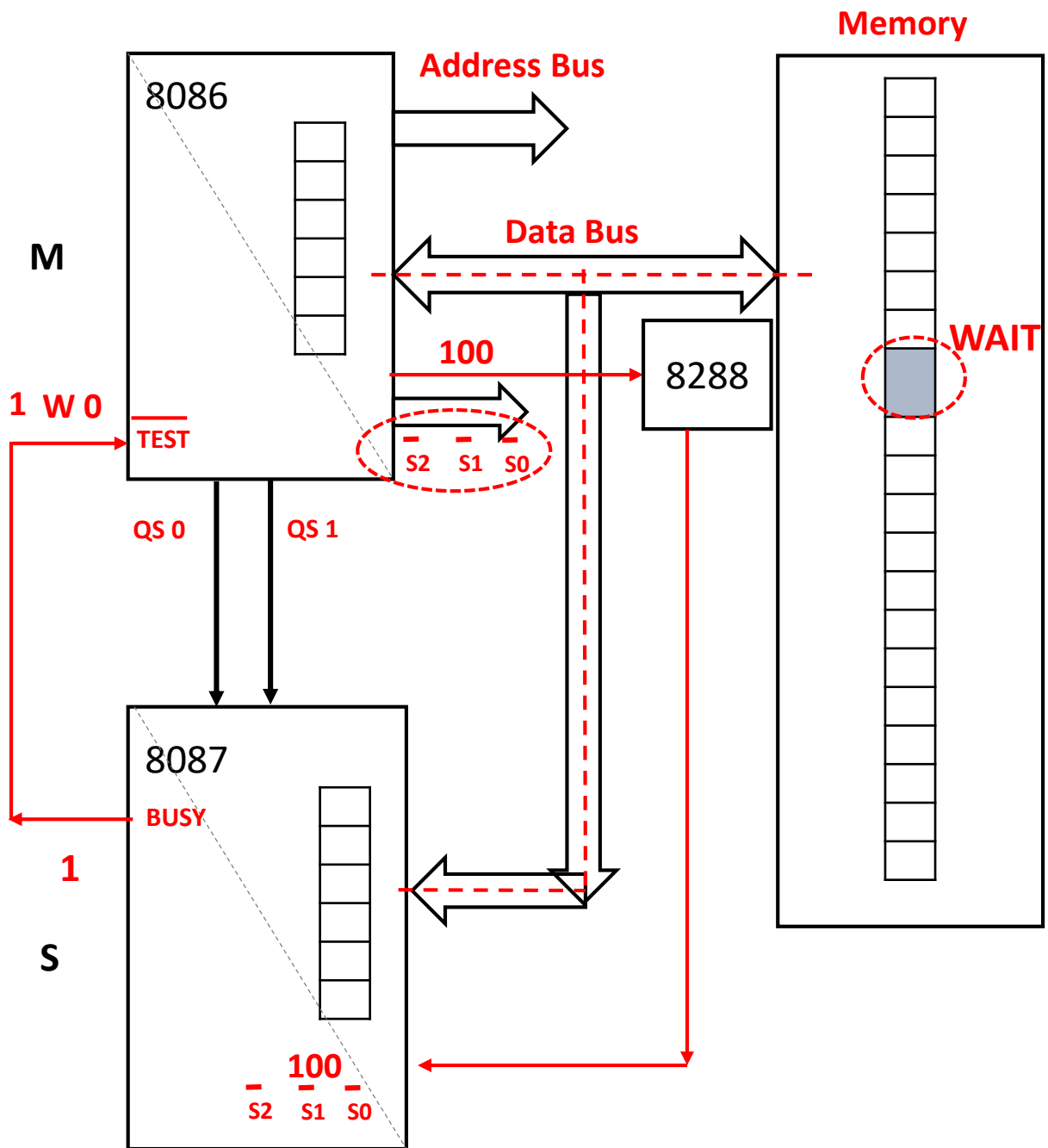
As a programmer are going to write WAIT instruction before 8087 instruction.

By executing wait instruction 8086 will check the TEST bar pin

If TEST bar = 1 , 8086 will wait until 8087 got free.

Once 8087 completes its own task it will send 0 on Busy pin

By receiving 0 on TEST bar, 8086 exits from wait state and fetch the next instruction.



Every instruction is fetched by 8086 and also received by 8087

It is not necessary that every data which is required to 8086 is also required to 8087

Bus carries both address and data

8087 must know whether the bus carries data or bus

When 8086 fetch the instruction it will send status signal 100 to 8288, which informs about the instruction fetch operation.

8288 will also send this status to 8087

QS0 – QS1 They are used to synchronised activity on both queue

QS1	QS0	
0	0	NOP
0	1	Remove Opcode
1	0	Clear Queue
1	1	Remove operand

-
- 1) As a co-processor (8087) is connected to 8086, 8086 operates in **Maximum Mode**.
Hence **MN/ $\overline{\text{MX}}$** is grounded.
 - 2) **8284** provides the common **CLK, RESET, READY** signals,
8282 are used to **latch the address**, **8286** are used as **data transreceivers**,
8288 generates **control signals** using S_2, S_1, S_0 as input from the currently active processor,
8259 PIC is used to accept the **interrupt from 8087** and send it to the μP .
 - 3) This interface is also called a **Closely Coupled Co-Processor configuration**.
Here **8086** is called as the **Host** and 8087 as **Co-Processor**, as it cannot operate all by itself.
 - 4) We write a **homogeneous program** which contains both 8086 as well as 8087 instructions.
 - 5) **Only 8086 can fetch instructions**, but these **instructions also enter 8087**.
8087 treats 8086 instructions as NOP.
 - 6) **ESC** is used as a **prefix for 8087 instructions**.
When an instruction with ESC prefix (5 MSB bits as **11011**) is encountered, 8087 is activated.
 - 7) The **ESC instruction** is **decoded by both** 8086 and 8087.
 - 8) If **ESC is present 8087 executes** the instruction and 8086 discards it.
If **ESC is not preset** then **8086 executes the instruction** and 8087 discards it.
 - 9) Once 8087 **begins execution** it makes the **BUSY o/p high** which is connected to the $\overline{\text{TEST}}$ of μP .
Now **8087 is executing its instruction** and **8086 moves ahead with its next instruction**.
Hence **MULTIPROCESSING takes place**. For doubts contact #BharatSir @9820408217
 - 10) **During execution**, if **8087 needs** to read/write more data (operands) from **the memory**, then it does so by **stealing bus cycles** from the μP in the following manner:
The $\overline{\text{RQ}} / \overline{\text{GT}}$ of 8087 is connected to $\overline{\text{RQ}} / \overline{\text{GT}}$ of the μP .
8087 gives an active low **Request pulse**.
8086 completes the current bus cycle and **gives** the **grant pulse** and **enters the Hold state**.
8087 uses the **shared system bus** to **perform the data transfer with the memory**.
8087 gives the **release pulse** and **returns the system bus** back to the μP .
 - 11) If **8086 requires** the result of the 8087 operation, it first **executes** the **WAIT** instruction.
WAIT makes the μP **check** the $\overline{\text{TEST}}$ pin.
If the **TEST pin is high** (**8087 is BUSY**), then the μP **enters WAIT state**
It **comes out** of it only **when $\overline{\text{TEST}}$ is low** (**8087 has finished its execution**).
Thus 8086 gets the correct result of an 8087 operation.
 - 12) During the execution if an **exception occurs**, which is **unmasked**, **8087 interrupts μP** using the **INT o/p pin** through the **PIC 8259**.

- 13) The QS_0 and QS_1 lines are used by 8087 to monitor the queue of 8086. 8087 needs to know when 8086 will decode the ESC instruction so it synchronizes its queue with 8086 using QS_0 and QS_1 as follows:

QS_1	QS_0	8087 Operation
0	0	NOP
0	1	8087 removes Opcode from Queue and compares 5 MSB bits with 11011.
1	0	8087 clears its queue.
1	1	8087 removes operand if earlier comparison with Opcode is successful.

This is the **complete inter-processor communication** between 8086 and 8087 to form a Homogeneous System.

FLOWCHART (Optional – only for understanding)

